bezierkit 软件包：Bézier 曲线输出工具

Pin Yue Sung[*†] Ling Tak Douglas Chung[*]

2026-10-04 · v1.0.0 · ·

目录

1 简介 2
1.1 符号 2
1.2 数学与证明 2
1.3 阅读指引 2
2 安装 3
2.1 系统需求 3
2.2 安装软件包 3
2.3 开发环境设置 3
3 快速开始 4
3.1 一条三次曲线 4
3.2 主要元件 4
4 几何数值对象 4
4.1 点与向量 5
4.2 点集合 5
4.3 参数 5
4.4 异常 5
5 Bézier 曲线 6
5.1 Bernstein 基底 6
5.2 曲线 7
5.3 求值 8
5.4 升阶 9
6 导数、反转与分割 9
6.1 导数 9
6.2 反转 10
6.3 分割 10
7 三次线段与路径 11
7.1 三次线段 11
7.2 紧密边界框 12
7.3 升阶 12
7.4 分段路径 13
7.5 连续性 14
8 建构与 Hermite 插值 15
8.1 端点建构 15
8.2 Hermite 插值 16
8.3 插值误差 17
9 拟合 18
9.1 自适应 Hermite 拟合 18
9.2 折线简化 20
10 等值线 21
10.1 等值集与切线 21
10.2 描绘 22
10.3 结果 24
10.4 限制 24
11 采样与导出 24
11.1 采样 24
11.2 四舍五入与坐标变换 24
11.3 JSON 25
11.4 SVG 25
11.5 TikZ 26
11.6 Matplotlib 26
12 命令行界面 26
12.1 控制点 27
12.2 命令 27
12.3 错误 27
A 证明 27
A.1 Bernstein 基底与求值 28
A.2 导数、反转与分割 30
A.3 三次线段与路径 31
A.4 建构与插值 33
A.5 拟合与等值线 34
A.6 导出 36
更新纪录 37
命令索引 38
参考文献 39

[*]台湾政治大学国际经营与贸易学系，台北，台湾。
[†]通讯作者。电子邮箱：contact@econ-viz.org

第 1 节　简介

bezierkit 是处理 Bézier 曲线的小型数学工具软件包。它可由控制点或端点条件构造曲线，计算曲线值与导数，分割与截取曲线，将曲线拟合到函数、采样点与等值线，并输出为 JSON、SVG 路径数据或 TikZ。软件包本身不绘图。Matplotlib、mosaickit 或 LaTeX 文档等绘图端接收精确的三次控制点，再负责绘制。

1.1　符号

点或向量位于某个维度 $d \geq 1$ 的 $\mathbb{R}^d$ 中；多数图使用 $d = 2$。$x \in \mathbb{R}^d$ 的欧氏范数为

$$\|x\| = \sqrt{x_1^2 + \ldots + x_d^2},$$

$\|x\|_\infty = \max_k |x_k|$ 为最大范数。集合 $S \subseteq \mathbb{R}^d$ 若对所有 $p, q \in S$ 与 $\lambda \in [0,1]$ 都有 $\lambda p + (1-\lambda) q \in S$，称为凸集；有限个点的凸包是它们所有凸组合 $\sum_i \lambda_i P_i$（$\lambda_i \geq 0$ 且 $\sum_i \lambda_i = 1$）的集合。映射 $A : \mathbb{R}^d \to \mathbb{R}^e$ 若可写成 $A(x) = Mx + v$（M 为矩阵，v 为向量），称为仿射映射。函数若有直到 k 阶的连续导数，称为 C^k。

n 次 Bézier 曲线有 $n+1$ 个控制点 $P_0, \ldots, P_n$，定义为映射

$$B(t) = \sum_{i=0}^{n} b_{i,n}(t) P_i, \quad t \in [0,1],$$

其中

$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

为 Bernstein 多项式（详见第 5 节）。依次连接控制点即为控制多边形。软件包中每条曲线的参数范围都是 $[0,1]$；超出范围的参数会抛出 `ParameterOutOfDomain`。符号 d 一律表示维度，容差写作 ε。

1.2　数学与证明

各章先回顾所用的标准定义并附出处，再以编号的引理、命题、定理与推论陈述算法所依据的性质：Bernstein 基底、de Casteljau 算法的求值与分割、Hermite 误差界，以及导出器的舍入误差。证明集中在附录 A，只想了解 API 时可略过。属于本软件包而不见于文献的约定，称为软件包约定。标准参考书为 Farin (2002) 与 Prautzsch et al. (2002)；Bernstein 基底另见 Farouki (2012)。

1.3　阅读指引

表 1
章节主题

主题	内容	章节
基础	点、向量、参数、异常	第 4 节
	Bernstein 基底、曲线、求值	第 5 节
	导数、反转、分割	第 6 节

主题	内容	章节
三次曲线	三次线段与分段路径	第 7 节
	构造与 Hermite 插值	第 8 节
近似	拟合函数与折线	第 9 节
	描绘等值线	第 10 节
输出	采样、JSON、SVG、TikZ、Matplotlib	第 11 节
	命令行	第 12 节

初次使用时，先读第 3 节与第 5 节。本手册的图本身就是 bezierkit 的输出：每条曲线都由第 11 节的 TikZ 导出器写出，再以 LaTeX 编译。

绘制本手册各图的脚本按主题分类，可一次下载：bezierkit-examples.zip。

第 2 节　安装

2.1　系统需求

bezierkit 需要 Python 3.10 以上版本与 NumPy。另有两组可选依赖：

`cli` `bezierkit` 命令（详见第 12 节）；安装 Typer 与 Rich。

`matplotlib` 与 Matplotlib `Path` 对象互相转换（详见第 11.6 节）。

2.2　安装软件包

```
uv add bezierkit                  # the library
uv add "bezierkit[cli]"           # with the command-line interface
uv add "bezierkit[matplotlib]"    # with the Matplotlib adapter
uv add "bezierkit==1.0.0"         # the version this manual describes
```

使用 pip 时，运行 `python -m pip install bezierkit`，可选依赖的写法相同。1.0.0 是第一个正式版：`bezierkit` 及其文档所列子包导出的名称按语义化版本管理，JSON 路径格式维持第 1 版。

2.3　开发环境设置

```
git clone https://github.com/EconViz/bezierkit.git
cd bezierkit
uv sync --all-extras --dev
uv run pytest
uv run ruff check src tests benchmarks
uv run lint-imports
```

持续整合流程在 Python 3.10、3.11、3.12 与 3.13 上运行相同命令。`lint-imports` 检查软件包分层：数学核心不导入任何绘图端，只有适配器导入 Matplotlib。

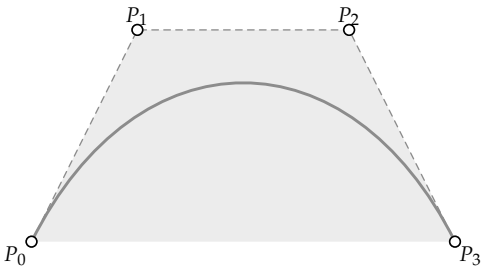


图 1
三次曲线及其控制凸包

第 3 节　快速开始

3.1　一条三次曲线

本手册通篇使用的曲线，是控制点为 $(0,0)$、$(1,2)$、$(3,2)$、$(4,0)$ 的三次曲线（参见图 1）。

例 1

```python
from bezierkit import BezierCurve, PiecewiseBezier, Point
from bezierkit.bezier import to_cubic
from bezierkit.export.tikz import to_tikz

curve = BezierCurve.cubic(Point(0, 0), Point(1, 2), Point(3, 2), Point(4, 0))
print(curve.at(0.5))                 # Point(coords=(2.0, 1.5))
print(curve.derivative().at(0.5))    # Point(coords=(4.5, 0.0))
left, right = curve.split(0.4)       # two cubics that together trace curve

path = PiecewiseBezier([to_cubic(curve)])
print(to_tikz(path, precision=2, options="thick"))
# \draw[thick] (0.00,0.00) .. controls (1.00,2.00) and (3.00,2.00) .. (4.00,0.00);
```

曲线由 P_0 出发、在 P_3 结束，离开 P_0 时朝向 P_1，抵达 P_3 时来自 P_2 的方向，且始终位于控制点的凸包内（详见推论 7）。最后一行是这条曲线的 TikZ 路径，可直接粘贴到 LaTeX 文档。

3.2　主要元件

- 数值对象：`Point`、`Vector` 与 `PointSet` 不可变，并检查维度（详见第 4 节）。
- 曲线：`BezierCurve` 可为任意次数与维度；`CubicBezierSegment` 是绘图端使用的三次曲线；`PiecewiseBezier` 将三次曲线连接成路径（详见第 5 节、第 7 节）。
- 构造：把端点条件、斜率、导数或 Hermite 数据转为控制点（详见第 8 节）。
- 近似：把平滑函数、采样点与等值线 $F(x,y)=c$ 转为误差已知的三次路径（详见第 9 节、第 10 节）。
- 输出：采样、JSON、SVG 路径数据、TikZ，以及 Matplotlib 路径（详见第 11 节）。

第 4 节　几何数值对象

所有数值对象都不可变，运算会返回新对象。坐标必须为有限值；创建对象时遇到 NaN 或无穷大会抛出 `ValueError`。

4.1　点与向量

`Point`　`Point(⟨浮点数⟩, ...)`

`Vector`　`Vector(⟨浮点数⟩, ...)`

$\mathbb{R}^d$ 中的点或向量，由 $d \geq 1$ 个坐标给定。点与向量按仿射几何的规则运算（参见表 2）。

表 2　点与向量的运算

表达式	结果
`point - point`	`Vector`
`point + vector`、`point - vector`	`Point`
`vector + vector`、`vector * scalar`	`Vector`；也可写成 `scalar * vector`

`coords`　坐标 tuple。

`dimension`　维度 d。

`x`　第一个坐标（存在时）。

`y`　第二个坐标（存在时）。

`z`　第三个坐标（存在时）。

`dot()`　两个向量的内积。

`norm()`　向量的欧氏长度。

`normalized()`　与向量同向的单位向量；零向量无法归一化。

维度不同时抛出 `DimensionMismatch`。`point + point` 不会被拒绝：它把坐标相加并返回 `Point`，只有在权重总和为一的组合中才有意义，例如 de Casteljau 算法中的平均。

```
from bezierkit import Point, Vector

p = Point(1, 2)
q = p + Vector(3, -1) * 2      # Point(coords=(7.0, 0.0))
v = q - p                      # Vector(coords=(6.0, -2.0))
print(v.norm())                # 6.324555320336759
```

4.2　点集合

`PointSet`　`PointSet(⟨控制点⟩)`

一批不可变的点，背后是只读的 (count, d) NumPy 数组，批量求值与采样都返回此对象。提供 `count`、`dimension`、`array`（只读副本）、各栏 `x`、`y`、`z`，可逐个取出 `Point`，也可用索引存取。

4.3　参数

`Interval`　`Interval(start, end)` 是闭区间，提供 `contains()`、`clamp()` 与 `linspace()`。

`ParameterValues`　由定义域检查单一参数或一维参数数组。每条曲线都使用它，因此对定义域为 $[0, 1]$ 的曲线调用 `at(1.2)` 会抛出 `ParameterOutOfDomain`。

4.4　异常

表 3　异常类

异常	抛出时机
`BezierKitError`	下列异常的基础类

异常	抛出时机
`DimensionMismatch`	组合不同维度的点、向量或线段
`DegreeError`	曲线次数不适用于某项运算，或没有控制点
`ParameterOutOfDomain`	参数超出曲线定义域 $[0,1]$
`ToleranceNotMet`	自适应拟合在限制内无法达到容差（详见第 9 节）

不属于几何的无效参数（例如负的容差）抛出 `ValueError`。

第 5 节　Bézier 曲线

5.1　Bernstein 基底

定义 1 (Bernstein 多项式) $n \geq 0$ 次 Bernstein 多项式为

$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad i = 0, \dots, n.$$

其他整数 i 令 $b_{i,n} = 0$。

这组多项式出自 Bernstein (1912)；Farouki (2012)回顾其历史与性质，三次的情形如图 2。以下证明用到两个二项式系数恒等式。

引理 2 (二项式恒等式) 令 $n \geq 1$，i、j 为整数，并约定 $k < 0$ 或 $k > m$ 时 $\binom{m}{k} = 0$。则

$$\binom{n-1}{j} + \binom{n-1}{j-1} = \binom{n}{j}$$

且

$$i\binom{n}{i} = n\binom{n-1}{i-1}.$$

命题 3 (Bernstein 基底) 对 $n \geq 0$，多项式 $b_{0,n}, \dots, b_{n,n}$ 是次数不超过 n 的实系数多项式向量空间的一组基底。

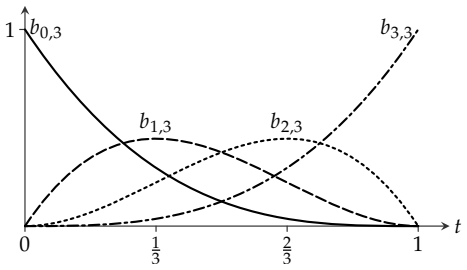


图 2
三次 Bernstein 多项式

这是 Floater (2025) 的定理 1.1；附录 A 从头证明。

命题 4 (单位分解) *对每个 $t \in [0,1]$，所有 $b_{i,n}(t) \geq 0$，且*

$$\sum_{i=0}^{n} b_{i,n}(t) = 1.$$

定义 5 (Bézier 曲线) 设 $P_0, \ldots, P_n \in \mathbb{R}^d$。以 $P_0, \ldots, P_n$ 为控制点的 n 次 Bézier 曲线为

$$B(t) = \sum_{i=0}^{n} b_{i,n}(t) P_i, \quad t \in [0,1],$$

其控制多边形为折线 $P_0 P_1 \ldots P_n$。

此定义依循 Bézier (1966)，形式如 Farin (2002) 与 Prautzsch et al. (2002)。

命题 6 (端点) *$b_{i,n}(0)$ 在 $i = 0$ 时为 1，其余为 0；$b_{i,n}(1)$ 在 $i = n$ 时为 1，其余为 0。因此 $B(0) = P_0$、$B(1) = P_n$。*

推论 7 (凸包) *每个 $B(t)$（$t \in [0,1]$）都位于控制点 $P_0, \ldots, P_n$ 的凸包内。*

命题 8 (仿射不变性) *对每个仿射映射 $A(x) = Mx + v$ 与每个 $t \in [0,1]$，*

$$A(B(t)) = \sum_{i=0}^{n} b_{i,n}(t) A(P_i).$$

变换曲线等同于变换其控制点。

推论 7 让曲线不超出控制多边形所张的区域；命题 8 则是导出器与 Matplotlib 适配器只需移动控制点，就能平移、缩放或旋转曲线的原因。

BernsteinBasis

```
BernsteinBasis(⟨整数⟩)(⟨浮点数⟩)
BernsteinBasis(⟨整数⟩).matrix(⟨数值⟩)
```

某一次数的基底，位于 `bezierkit.bezier.basis`。在 t 调用时，以数组返回 $i = 0, \ldots, n$ 的 $b_{i,n}(t)$；`matrix()` 对每个参数值返回一列。

5.2　曲线

BezierCurve

```
BezierCurve(⟨控制点⟩, *, evaluator=None)
BezierCurve.linear(p0, p1)
BezierCurve.quadratic(p0, p1, p2)
BezierCurve.cubic(p0, p1, p2, p3)
```

不可变的 Bézier 曲线，次数 $n \geq 0$、维度 $d \geq 1$ 均不限，参数范围为 $[0,1]$。`points` 可为 `Point` 序列、`PointSet`、$(n+1) \times d$ 数组或 `ControlPolygon`；所有控制点必须同一维度。

`degree`	次数 n。
`dimension`	维度 d。
`control_points`	以 `PointSet` 表示的控制点。
`at()`	以 `Point` 返回 $B(t)$；曲线对象也可直接调用，`curve(t)`。
`at_many()`	以 `PointSet` 返回多个参数下的值。
`evaluator`	求值策略（见下文）。

曲线的运算 `derivative()`、`split()`、`segment()` 与 `reversed()` 详见第 6 节。

5.3　求值

de Casteljau 算法以反复的线性插值计算 $B(t)$。

定义 9 (de Casteljau 点) 对参数 t，令 $P_i^{(0)} = P_i$，并对 $r = 1, \dots, n$ 令

$$P_i^{(r)} = (1-t)P_i^{(r-1)} + tP_{i+1}^{(r-1)}, \quad i = 0, \dots, n-r.$$

每一轮对相邻点取平均，多边形少一个点；n 轮后只剩一点（参见图 3）。此算法源自 Paul de Casteljau 约 1959 年在 Citroën 的工作 (Müller, 2024)。

定理 10 (de Casteljau) 对 $0 \leq r \leq n$ 与 $0 \leq i \leq n-r$，

$$P_i^{(r)} = \sum_{j=0}^{r} b_{j,r}(t) P_{i+j}.$$

特别地，$P_0^{(n)} = B(t)$。

此叙述见 Floater (2025) 的定理 1.6 与 1.7。

每个中间点都是控制点的凸组合，算法不会产生互相抵消的大数，因此在高次数时数值稳定。每个参数的计算量为 $O(n^2)$。

`DeCasteljauEvaluator`　默认的求值策略，位于 `bezierkit.bezier.evaluation`：以 NumPy 一次对整批参数运行上述算法。

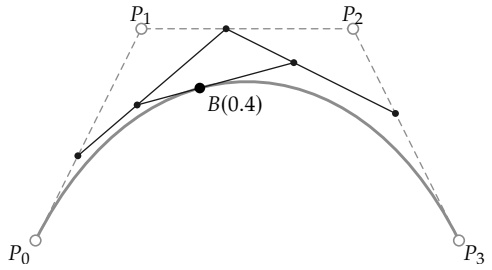


图 3
de Casteljau 算法

`BernsteinEvaluator` 以 Bernstein 矩阵乘上控制点。次数低、批量大时较快，但高次数时会求和可能互相抵消的项。根据定理 10，两种策略计算的是同一个多项式。

```
from bezierkit import BezierCurve
from bezierkit.bezier.evaluation import BernsteinEvaluator

fast = BezierCurve(curve.control_points,
evaluator=BernsteinEvaluator())
print(fast.at(0.5))   # Point(coords=(2.0, 1.5)), as with the default
```

仓库的 `benchmarks/` 目录比较两者在 400、10,000 与 100,000 个参数值下的效能。

5.4　升阶

n 次曲线也是 $n+1$ 次曲线；新的控制点是旧控制点中相邻两点的凸组合。

引理 11 (升阶恒等式) *对 $n \geq 0$、$0 \leq i \leq n$ 与每个 t，*

$$b_{i,n}(t) = \frac{n+1-i}{n+1} b_{i,n+1}(t) + \frac{i+1}{n+1} b_{i+1,n+1}(t).$$

命题 12 (升阶) *设 B 的控制点为 $P_0, \dots, P_n$。对 $k = 0, \dots, n+1$ 令*

$$Q_k = \frac{k}{n+1} P_{k-1} + \left(1 - \frac{k}{n+1}\right) P_k,$$

其中 P_{-1} 与 P_{n+1} 可任意选取，因为它们的系数为零。则

$$B(t) = \sum_{k=0}^{n+1} b_{k,n+1}(t)\, Q_k \quad \text{对所有 } t.$$

曲线及其参数化都不变。

此公式是标准结果 (Farin, 2002; Prautzsch et al., 2002)；附录 A 的证明不依赖它们。命题 22 把它用在直线与二次曲线，也就是本软件包需要的情形。

第 6 节　导数、反转与分割

每项运算只作用于控制点，并返回沿用原曲线求值策略的新曲线。

6.1　导数

Bézier 曲线的导数 $B'(t)$ 是其坐标函数（t 的多项式）的导数，为 $\mathbb{R}^d$ 中的向量。

引理 13 (Bernstein 多项式的导数) *对 $n \geq 1$ 与每个整数 i，*

$$b'_{i,n}(t) = n\big[b_{i-1,n-1}(t) - b_{i,n-1}(t)\big].$$

此为 Floater (2025) 的引理 1.4。

定理 14 (速端曲线) *n 次 Bézier 曲线（$n \geq 1$）的导数是 $n-1$ 次 Bézier 曲线*

$$B'(t) = \sum_{i=0}^{n-1} b_{i,n-1}(t) \cdot n(P_{i+1} - P_i).$$

控制点为 $n(P_{i+1} - P_i)$ 的曲线称为 B 的速端曲线 (Floater (2025) 的定理 1.8; 另见 Farin (2002))。

推论 15 (端点切线) $B'(0) = n(P_1 - P_0)$，$B'(1) = n(P_n - P_{n-1})$。

这说明了曲线为何朝 P_1 的方向离开 P_0，并从 P_{n-1} 的方向抵达 P_n。

`BezierCurve.derivative`

`derivative(order=1)`

以 `BezierCurve` 返回 `order` 阶导数，即套用定理 14 `order` 次。常数（0 次）的导数是 0 次的零曲线；`order=0` 返回曲线本身。

```
d = curve.derivative()
print(list(d.control_points))
# [Point(coords=(3.0, 6.0)), Point(coords=(6.0, 0.0)),
Point(coords=(3.0, -6.0))]
print(d.at(0.5))   # Point(coords=(4.5, 0.0)): the tangent at the top
is horizontal
```

6.2　反转

引理 16 (对称性) *对所有 i 与 t，$b_{i,n}(1-t) = b_{n-i,n}(t)$。*

命题 17 (反转) *控制点为 $P_n, \ldots, P_0$ 的曲线就是 $t \mapsto B(1-t)$。*

`BezierCurve.reversed`

根据命题 17，返回反向遍历的曲线。

6.3　分割

在 $t = c$ 运行 de Casteljau 算法不只得到曲线值：每一轮的第一个点构成 c 之前那段曲线的控制多边形，最后一个点构成 c 之后那段的控制多边形（参见图 4）。

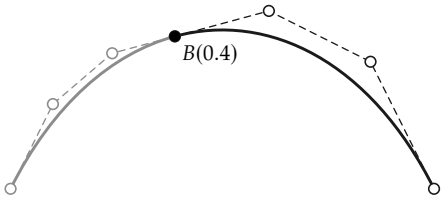


图 4
在 $t = 0.4$ 分割曲线

定理 18 (分割) 令 $c \in [0,1]$，并在 $t = c$ 计算定义 9 的各点。令 $L_j = P_0^{(j)}$、$R_j = P_j^{(n-j)}$，$j = 0, \ldots, n$。则对每个 $s \in [0,1]$

$$B(cs) = \sum_{j=0}^{n} b_{j,n}(s) L_j$$

且

$$B(c + (1-c)s) = \sum_{j=0}^{n} b_{j,n}(s) R_j.$$

这是标准结果，见 Floater (2025)（第 8.4 节，由 blossom 导出）与 Farin (2002)。

`BezierCurve.split` `split(⟨浮点数⟩)`

`BezierCurve.segment` `segment(t0, t1)`

`split(c)` 返回定理 18 的两条曲线，次数不变，参数范围都是 $[0,1]$。`segment(t0, t1)` 返回曲线在 t_0 与 t_1 之间的部分，重新参数化到 $[0,1]$；$t_0 = t_1$ 时返回单点 $B(t_0)$，即 0 次曲线。参数超出 $[0,1]$ 或 $t_0 > t_1$ 时抛出异常。

`segment()` 分割两次：先在 t_1 分割并保留左段，再在 t_0/t_1 分割该段并保留右段。

推论 19 (截取) 对 $0 \le t_0 < t_1 \le 1$，*`segment(t0, t1)` 返回的曲线为* $s \mapsto B(t_0 + (t_1 - t_0)s)$。

```
left, right = curve.split(0.4)
print(left.at(1.0), right.at(0.0))          # both B(0.4) = (1.552,
1.44)
print(curve.segment(0.25, 0.75).at(0.5))    # B(0.5) = (2.0, 1.5)
```

第 7 节　三次线段与路径

绘图端画的是三次曲线：TikZ 的 `.. controls ..`、SVG 的 `C` 与 Matplotlib 的 `CURVE4` 都接收四个控制点。`CubicBezierSegment` 就是以数值对象表示的三次曲线，`Piecewise Bezier` 则把三次曲线连接成路径。

7.1　三次线段

`CubicBezierSegment` `CubicBezierSegment(p0, p1, p2, p3, *, fit_error=None)`

不可变的三次曲线, 四个控制点即字段 p0、p1、p2、p3（亦可用 control_points 或逐一取出）。求值、微分、分割、截取与反转的方式与 BezierCurve 相同, 但 split()、segment() 与 reversed() 仍回传三次线段。segment(t, t) 回传四个控制点都是 $B(t)$ 的退化三次曲线。

fit_error　拟合线段的实测误差（详见第 9 节）；分割、截取与反转后保留，判断相等时忽略。

float | *None* · 默认 None

as_curve()　转为 3 次 BezierCurve，不改变任何控制点。

from_curve(curve)　由 3 次 BezierCurve 转换，不改变任何控制点。

from_line()　直线的精确三次表示（详见命题 22）。

from_quadratic()　二次曲线的精确三次表示（详见命题 22）。

7.2　紧密边界框

定义 20 (轴对齐边界框) 非空有界集合 $K \subset \mathbb{R}^d$ 的轴对齐边界框，是包含 K 的最小方框

$$[\alpha_1, \beta_1] \times \ldots \times [\alpha_d, \beta_d],$$

其中 $\alpha_k = \inf\{x_k : x \in K\}$、$\beta_k = \sup\{x_k : x \in K\}$。

控制点凸包包含曲线（详见推论 7），但可能比曲线大得多。bounding_box 回传曲线本身的定义 20 所定义的方框（参见图 5），其候选点来自各坐标导数的根。

命题 21 (三次曲线的边界框) 设 B 是控制点为 $P_0, \ldots, P_3 \in \mathbb{R}^d$ 的三次曲线，令

$$a = -P_0 + 3P_1 - 3P_2 + P_3, \quad b = 3P_0 - 6P_1 + 3P_2, \quad c = 3(P_1 - P_0).$$

则 $B(t) = at^3 + bt^2 + ct + P_0$。对每个坐标 k，令 T_k 由 0、1 与下式在 $(0,1)$ 内的根组成

$$B_k'(t) = 3a_k t^2 + 2b_k t + c_k$$

（此多项式不恒为零时）；若它恒为零，则 $T_k = \{0,1\}$。于是 $B([0,1])$ 的边界框坐标为

$$\alpha_k = \min_{t \in T_k} B_k(t), \quad \beta_k = \max_{t \in T_k} B_k(t).$$

CubicBezierSegment.bounding_box　回传 (low, high) 两点，分别为曲线在各轴的最小值与最大值，由命题 21 的候选参数求值而得。

7.3　升阶

升阶（详见命题 12）把 n 次曲线写成参数化相同的 $n+1$ 次曲线。本套件用它把直线与二次曲线写成三次曲线，使每个绘图端都收到四个控制点。对命题 12 套用一次或两次，即得下列控制点。

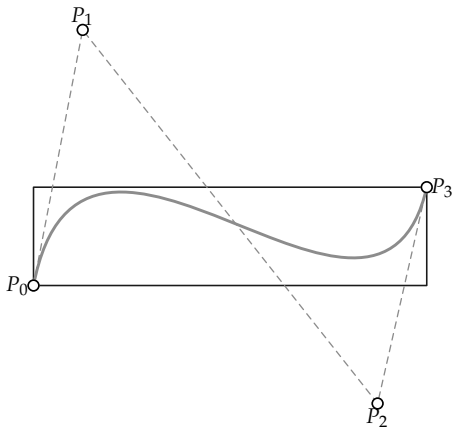


图 5
三次曲线的紧密边界框

命题 22 (直线与二次曲线的三次表示) 由 P_0 到 P_3、参数化为 $(1-t)P_0 + tP_3$ 的直线，是控制点为

$$P_0,\quad P_0 + \frac{1}{3}(P_3 - P_0),\quad P_0 + \frac{2}{3}(P_3 - P_0),\quad P_3$$

的三次曲线。控制点为 P_0, C, P_3 的二次曲线，是控制点为

$$P_0,\quad P_0 + \frac{2}{3}(C - P_0),\quad P_3 + \frac{2}{3}(C - P_3),\quad P_3$$

的三次曲线。两种情形下曲线与其参数化都不变。

`to_cubic` `to_cubic(⟨曲线⟩)`
位于 `bezierkit.bezier`。依命题 22 将一次、二次或三次 `BezierCurve` 转为 `CubicBezierSegment`（传入线段时原样回传）；更高次数抛出 `DegreeError`。

`line_to_cubic` `line_to_cubic(p0, p3)`
由 `p0` 到 `p3` 的直线的三次表示。

`quadratic_to_cubic` `quadratic_to_cubic(p0, control, p3)`
控制点如给定的二次曲线的三次表示。

```python
from bezierkit.bezier import to_cubic

q = BezierCurve.quadratic(Point(0, 0), Point(3, 6), Point(9, 0))
print(to_cubic(q).control_points)
# (Point(coords=(0.0, 0.0)), Point(coords=(2.0, 4.0)),
#  Point(coords=(5.0, 4.0)), Point(coords=(9.0, 0.0)))
```

7.4　分段路径

`PiecewiseBezier` `PiecewiseBezier(⟨segments⟩, *, closed=False, continuity_tolerance=1e-9)`

`PiecewiseBezier.compound(⟨paths⟩)`

由三次线段组成的路径。相邻线段必须相接：每段的 `p3` 与下一段的 `p0` 距离不得超过 `continuity_tolerance`，封闭路径最后的 `p3` 与第一个 `p0` 亦然，否则抛出 `ValueError`。`closed` 是给导出器的信息（SVG 的 `Z`、TikZ 的 `cycle`），不会额外加入封闭线段。

本套件以均匀方式将路径参数化。这是本套件的约定，并非通用概念：由 m 段 $S_0, \dots, S_{m-1}$ 组成的路径是映射

$$P(t) = S_k(mt - k), \quad k = \min(\lfloor mt \rfloor, m-1), \quad t \in [0,1].$$

不论长短，每段都占 $[0,1]$ 的 $1/m$。

`segments`	依序排列的线段。
`control_points`	其控制点。
`subpaths`	各 `BezierSubpath`。
`at(t)`	依均匀参数化求值。
`at_many(values)`	在多个参数上求值。
`split(t)`	在 `t` 分割路径；参数落在某段内时会分割该段（详见定理 18）。
`segment(t0, t1)`	保留两个参数之间的部分。
`reversed()`	反转线段顺序与每一段。
`closed`	封闭旗标。
`is_compound`	是否含多条子路径。
`dimension`	维度 d。

```
from bezierkit import CubicBezierSegment, PiecewiseBezier

path = PiecewiseBezier([
    CubicBezierSegment.from_line(Point(0, 0), Point(2, 0)),
    CubicBezierSegment.from_line(Point(2, 0), Point(2, 4)),
])
print(path.at(0.75))                      # Point(coords=(2.0, 2.0))
print(path.segment(0.25, 0.75).at(1.0))   # Point(coords=(2.0, 2.0))
```

`compound()` 把多条路径合为一条，各子路径保持分开，例如有洞的形状。复合路径没有单一的参数化：求值、分割或截取会抛出 `ValueError`；反转与导出则可正常使用。分割或截取封闭路径也会抛出异常（整条路径或单点除外），因为封闭路径没有可保留的起点与终点。

7.5　连续性

两条曲线在接点相接。设 $h_S, h_T > 0$，$S : [u_0 - h_S, u_0] \to \mathbb{R}^d$ 与 $T : [u_0, u_0 + h_T] \to \mathbb{R}^d$ 具有所需阶数的连续导数（区间端点取单边导数），C 为在前一区间等于 S、在后一区间等于 T 的曲线。

定义 23 (参数连续性) 对 $k \geq 0$，若 $S^{(j)}(u_0) = T^{(j)}(u_0)$ 对 $j = 0, \dots, k$ 成立，称曲线 C 在 u_0 为 C^k。

定义 24 (几何连续性 G^1) 若 $S(u_0) = T(u_0)$，切矢量 $S'(u_0)$ 与 $T'(u_0)$ 皆非零，且存在 $\lambda > 0$ 使 $T'(u_0) = \lambda S'(u_0)$，称曲线 C 在 u_0 为 G^1；也就是两侧的单位切矢量相同。

参数连续性与几何连续性是连接曲线的标准概念 (Farin, 2002; Prautzsch et al., 2002)；任意阶 k 的几何连续性见 Barsky & DeRose (1989)。本手册只用到 C^0、C^1 与 G^1。切矢量非零的 C^1 接点必为 G^1。

命题 25 (三次曲线接点的连续性) 设 S、T 是控制点分别为 $P_0,\dots,P_3$ 与 $Q_0,\dots,Q_3$ 的三次曲线，并分别在 $(u-u_0+h_S)/h_S$ 与 $(u-u_0)/h_T$ 求值。

1. C 在 u_0 为 C^0，若且唯若 $P_3 = Q_0$。
2. 若 $P_3 = Q_0$，则 C 在 u_0 为 C^1，若且唯若

$$\frac{P_3 - P_2}{h_S} = \frac{Q_1 - Q_0}{h_T}.$$

3. 若 $P_3 = Q_0$、$P_3 \neq P_2$ 且 $Q_1 \neq Q_0$，则 C 在 u_0 为 G^1，若且唯若存在 $\mu > 0$ 使 $Q_1 - Q_0 = \mu(P_3 - P_2)$。
4. 在 `PiecewiseBezier` 的均匀参数化下 $h_S = h_T$，因此第 2 项的条件即为 $P_3 - P_2 = Q_1 - Q_0$。

`PiecewiseBezier` 只要求 C^0，也就是端点相接。接点是否也要平滑由调用端决定：无异曲线的折角或折线的转角本来就该保留。

第 8 节　建构与 Hermite 插值

三次曲线由两个端点及其导数决定。本章的建构方法把这类端点条件转为控制点，经济学中只知道几个数值与斜率的曲线，就是这样变成 Bézier 曲线。

8.1　端点建构

每种建构都是带有 `build()` 方法的不可变 dataclass，位于 `bezierkit.construction`。

`EndpointDerivatives` `EndpointDerivatives(start, end, start_derivative, end_derivative).build()`
满足 $B(0)=$ `start`、$B(1)=$ `end`、$B'(0)=$ `start_derivative`、$B'(1)=$ `end_derivative` 的三次曲线，维度不限。

命题 26 (端点条件) 设 $P_0, P_3, D_0, D_1 \in \mathbb{R}^d$。控制点为

$$P_0,\quad P_0 + D_0/3,\quad P_3 - D_1/3,\quad P_3$$

的三次 *Bézier* 曲线满足 $B(0)=P_0$、$B(1)=P_3$、$B'(0)=D_0$、$B'(1)=D_1$，且它是唯一满足这些条件、次数不超过 3 的多项式曲线。

`TangentDirections` `TangentDirections(start, end, start_direction, end_direction, start_handle=1.0, end_handle=1.0).build()`

以方向给定端点切线，控制柄长度为 $|P_1 - P_0| =$ `start_handle`、$|P_3 - P_2| =$ `end_handle`。方向会先范式，因此只有方向有影响；控制柄长度必须为有限的非负值。依命题 26，$B'(0)$ 等于 3 倍 `start_handle` 乘以起点的单位方向。

`PlanarSlopes` `PlanarSlopes(start, end, start_slope, end_slope, start_handle=1.0, end_handle=1.0).build()`

两端斜率 $\mathrm{d}y/\mathrm{d}x$ 为给定值的平面三次曲线，以方向 $(1, m)$ 的 `TangentDirections` 建构。斜率必须为有限值；垂直切线请改用 `TangentDirections`。

```
from bezierkit.construction import PlanarSlopes

demand = PlanarSlopes(
    start=Point(0, 5), end=Point(5, 0), start_slope=-2, end_slope=-0.3,
).build()
print(demand.control_points[1])   # (0.447, 4.106): one unit from (0, 5) along slope -2
```

8.2　Hermite 插值

三次 Hermite 插值在区间两端同时吻合函数值与导数。

定义 27 (三次 Hermite 插值多项式) 设 f 在 $[x_0, x_1]$ 上可微，$x_0 \neq x_1$。f 的三次 Hermite 插值多项式是次数不超过 3、满足

$$H(x_i) = f(x_i), \quad H'(x_i) = f'(x_i), \quad i = 0, 1$$

的多项式 H。对曲线则逐一插值每个坐标。

这是 Burden & Faires (2011) 第 3.4 节中 Hermite 多项式取两个节点的情形，该书的定义 3.8。这样的多项式存在且唯一：

命题 28 (存在性与唯一性) *定义 27 的三次 Hermite 插值多项式存在且唯一。令 $h = x_1 - x_0$、$f_i = f(x_i)$、$m_i = f'(x_i)$，则*

$$H(x) = \sum_{i=0}^{3} b_{i,3}((x - x_0)/h) c_i,$$

其中系数为

$$c_0 = f_0, \quad c_1 = f_0 + h m_0/3, \quad c_2 = f_1 - h m_1/3, \quad c_3 = f_1.$$

任意节点数的同一结论，见 Stoer & Bulirsch (2002) 的定理 (2.1.5.2)。在宽度为 $h = t_1 - t_0$ 的参数区间 $[t_0, t_1]$ 上，Bézier 线段在 $s = (t - t_0)/h$ 处求值，导数因此乘上 h。

`parametric_hermite` `parametric_hermite(p0, p3, derivative0, derivative1, *, t0=0.0, t1=1.0)`

控制点为 P_0、$P_0 + hD_0/3$、$P_3 - hD_1/3$、P_3 的三次线段，位于 `bezierkit.interpolation`。$h = 0$ 时抛出 `ValueError`；h 为负表示反向的区间。

命题 29 (Hermite 插值) 对 *parametric_hermite* 回传的线段，令 $H(t) = B((t-t_0)/h)$。则 $H(t_0) = P_0$、$H(t_1) = P_3$、$H'(t_0) = D_0$、$H'(t_1) = D_1$。

graph_hermite

`graph_hermite(*, x0, x1, y0, y1, m0, m1)`

由函数值 $y_0 = f(x_0)$、$y_1 = f(x_1)$ 与斜率 $m_0 = f'(x_0)$、$m_1 = f'(x_1)$ 得到图形 $y = f(x)$ 的插值曲线，即取 $P = (x, f(x))$、$D = (1, f'(x))$、$t = x$ 的参数形式。控制点为

$$(x_0, y_0),\quad \left(x_0 + \frac{h}{3}, y_0 + hm_0/3\right),\quad \left(x_1 - \frac{h}{3}, y_1 - hm_1/3\right),\quad (x_1, y_1),$$

其中 $h = x_1 - x_0$（参见图 6）。

命题 30 (图形形式) *graph_hermite* 的线段是 f 在 $[x_0, x_1]$ 上的三次 *Hermite* 插值多项式 H 的图形：即 $s \mapsto (x_0 + hs, H(x_0 + hs))$，$s \in [0,1]$。

```
from bezierkit.interpolation import graph_hermite

# f(x) = 4 / x^2 on [1, 1.5]
segment = graph_hermite(x0=1, x1=1.5, y0=4, y1=16 / 9, m0=-8, m1=-64 /
27)
```

8.3　插值误差

定理 31 (Hermite 误差界限) 设 f 在 $[x_0, x_1]$ 上四阶连续可微，$x_0 < x_1$，H 为其三次 *Hermite* 插值多项式（详见定义 27），令 $h = x_1 - x_0$、$M = \max|f^{(4)}|$。对每个 $x \in [x_0, x_1]$，存在 $\xi \in [x_0, x_1]$ 使

$$f(x) - H(x) = \frac{f^{(4)}(\xi)}{24}(x - x_0)^2 (x - x_1)^2,$$

因此

$$\max_{x \in [x_0, x_1]} |f(x) - H(x)| \le \frac{Mh^4}{384}.$$

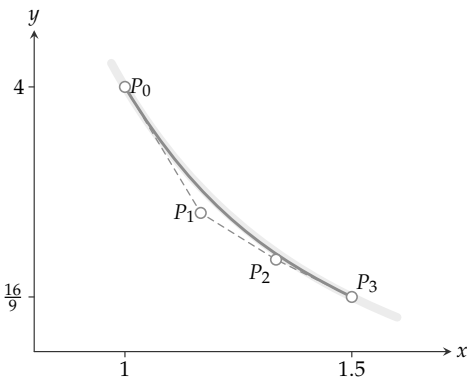


图 6
$4/x^2$ 的 Hermite 插值

误差公式见 Burden & Faires (2011) 的定理 3.9（其证明留作该书习题 3.4.11）与 Stoer & Bulirsch (2002) 的定理 (2.1.5.9)。此界限是锐利的：对 $[0,1]$ 上的 $f(x)=x^4$，误差为 $x^2(x-1)^2$，于 $x=1/2$ 的最大值 $1/16$ 等于 $M=24$ 时的 $Mh^4/384$。

区间减半时界限变为 $1/16$，自适应拟合正是利用这一点（详见第 9 节）。以 $f(x)=4/x^2$ 在 $[1,1.5]$ 为例，$f^{(4)}(x)=480/x^6$，故 $M=480$，界限为 $480\cdot 0.5^4/384\approx 0.078$。

`hermite_error_bound` `hermite_error_bound(max_fourth_derivative, t0, t1)`
定理 31 的界限 $M\,|t_1-t_0|^4\,/384$，位于 `bezierkit.fitting`。M 必须为有限的非负值。

此界限适用于纯量函数。对曲线 $t\mapsto(x(t),y(t))$，分别套用到每个坐标。在 $\mathbb{R}^d$ 中，欧氏误差至多为最大坐标界限的 $\sqrt{d}$ 倍，因为对 $v\in\mathbb{R}^d$ 有 $\|v\|_2\le\sqrt{d}\|v\|_\infty$（见 Golub & Van Loan (2013) 第 2.2 节）。

第 9 节　拟合

拟合把平滑函数或一串采样点转为 `PiecewiseBezier`，并逐段量测、回报误差，单位与曲线坐标相同。相关函数位于 `bezierkit.fitting`。

9.1　自适应 Hermite 拟合

参数曲线 $C:[t_0,t_1]\to\mathbb{R}^d$ 在区间 $[a,b]$ 上的拟合，是逐一坐标取定义 27 的三次 Hermite 插值多项式（详见命题 29）。套件在 q 个等距的量测参数 $\tau_j=a+j(b-a)/(q-1)$（$j=0,\dots,q-1$，q 为 `error_samples`）上量测误差：

$$e=\max_{0\le j\le q-1}\bigl\|C(\tau_j)-H(\tau_j)\bigr\|_2.$$

此实测误差是本套件的约定，绝不大于线段真实的最大误差。

`fit_parametric` `fit_parametric(⟨function⟩, ⟨derivative⟩, *, t0, t1, tolerance, max_depth=20, max_segments=4096, error_samples=9)`
在 $[t_0,t_1]$ 上拟合参数曲线 $C(t)$；`function` 回传 `Point`，`derivative` 回传 `Vector`。算法为递归二分：

1. 以目前区间两端的 C 与 C' 创建 Hermite 线段（详见命题 29）。
2. 量测误差 e。
3. e 不超过 `tolerance` 时保留此线段，并将 e 记为其 `fit_error`；否则将区间对半，两半各自拟合。

因此每个保留的线段在量测参数上都符合容许误差。若区间到了 `max_depth` 仍不合格，或保留它会超过 `max_segments`，就抛出 `ToleranceNotMet`，而不回传品质较差的路径。

`tolerance`
float
容许误差，单位同坐标；必须为正。

`max_depth`
int · 默认 20
起始区间最多可对半的次数。

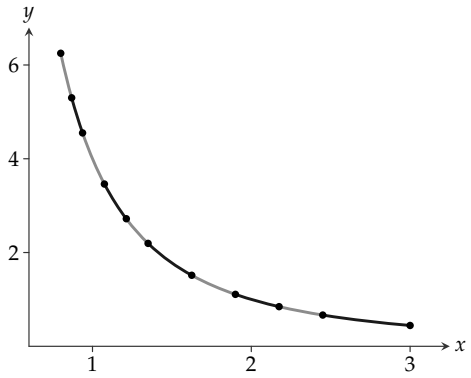


图 7
$4/x^2$ 的自适应拟合

`max_segments`
int · 默认 4096

结果最多的线段数。

`error_samples`
int · 默认 9

每段的量测参数个数，至少 3。

`fit_graph`

`fit_graph(`⟨*function*⟩`, `⟨*derivative*⟩`, *, x0, x1, tolerance, ...)`

拟合图形 $y = f(x)$，$x \in [x_0, x_1]$：即以 $C(x) = (x, f(x))$、$C'(x) = (1, f'(x))$ 调用 `fit_parametric`，选项相同，误差为 (x, y) 平面上的欧氏距离。

```
from bezierkit.fitting import fit_graph

path = fit_graph(lambda x: 4 / x**2, lambda x: -8 / x**3,
                 x0=0.8, x1=3, tolerance=1e-3)
print(len(path.segments))                    # 10
print(max(s.fit_error for s in path))        # 0.000676
```

曲线弯曲剧烈处线段较短，接近直线处线段较长（参见图 7，相邻线段以不同颜色区分）。Hermite 误差界限可预估二分需要的深度：

推论 32 (自适应拟合的深度) *设 C 的每个坐标在 $[t_0, t_1]$ 上四阶连续可微且 $|C_k^{(4)}| \le M$，令 $L = |t_1 - t_0|$，d 为维度，$\varepsilon > 0$ 为容许误差。令*

$$k^{*} = \max\!\left(0, \left\lceil \frac{1}{4} \log_2\!\left(\sqrt{d}\, M \frac{L^4}{384\varepsilon} \right) \right\rceil\right),$$

$M = 0$ 时 $k^{*} = 0$。深度 $k \ge k^{*}$ 的每个区间都能通过误差检查。因此若 `max_depth` $\ge k^{*}$ 且 `max_segments` $\ge 2^{k^{*}}$，`fit_parametric` 不会抛出 `ToleranceNotMet`，也不会产生深度大于 k^{*} 的区间。

实测误差是线段真实最大误差的下界，因为只在量测参数上检查。若曲线可能在量测点之间起伏，请增加 `error_samples`。

9.2　折线简化

定义 33 (点到线段的距离) 对 $x, a, b \in \mathbb{R}^d$，x 到线段 $[a, b]$ 的距离为

$$\operatorname{dist}(x, [a,b]) = \min_{\lambda \in [0,1]} \|x - a - \lambda(b-a)\|_2.$$

以递归分割简化折线，就是 Ramer–Douglas–Peucker 算法 (Douglas & Peucker, 1973; Ramer, 1972)。`fit_polyline` 全程使用定义 33 的距离。

`fit_polyline` `fit_polyline(⟨控制点⟩, *, tolerance, closed=False, duplicate_tolerance=1e-12, preserve_corners=True, corner_angle=pi/4)`

把采样点简化为由直线弦组成的路径，每条弦以精确的三次曲线表示（详见命题 22）。步骤如下：

1. 删除与前一点距离在 `duplicate_tolerance` 以内的点。封闭折线另外删除结尾重复的起点，并旋转为从字典序最小的点开始，使结果与采样起点无关。
2. 保留两端点；激活 `preserve_corners` 时，另保留方向转折至少 `corner_angle` 弧度的顶点。
3. 在相邻的保留顶点之间套用递归步骤：若离弦最远的顶点距离不超过 `tolerance`，以弦取代整段；否则保留该顶点，两侧递归处理。

每条弦的 `fit_error` 是被它取代的顶点到弦的最大距离（参见图 8）。

递归最多有与顶点数相同的层数，每层对每个顶点至多扫描一次，因此最坏情况下与顶点数成平方关系。Hershberger & Snoeyink (1994) 给出此算法的 $O(n \log n)$ 实作。

命题 34 (折线容许误差) *设 $v_0, \ldots, v_N$ 为通过步骤 1 的顶点（封闭折线有 $v_N = v_0$），$0 = a_0 < \ldots < a_m = N$ 为输出保留的顶点指针，ε 为 `tolerance`。对每个 j 与每个满足 $a_j \le i \le a_{j+1}$ 的 i，*

$$\operatorname{dist}\Big(v_i, \big[v_{a_j}, v_{a_{j+1}}\big]\Big) \le \varepsilon.$$

特别地，每个输出线段的 `fit_error` 都不超过 ε。

推论 35 (与折线的偏差) *沿用命题 34 的记号：*

1. *折线 $v_0 v_1 \ldots v_N$ 上的每个点，都与输出的某条弦 $\big[v_{a_j}, v_{a_{j+1}}\big]$ 相距不超过 ε。*
2. *每个输入点与某条弦相距不超过 $\varepsilon + \delta$，其中 δ 为 `duplicate_tolerance`；通过步骤 1 的点相距不超过 ε。*

`maximum_polyline_deviation` `maximum_polyline_deviation(⟨控制点⟩, ⟨路径⟩)`

各点到路径中最近之弦 P_0P_3 的最大距离：可用来检查命题 34 与推论 35，也涵盖步骤 1 删除的点。

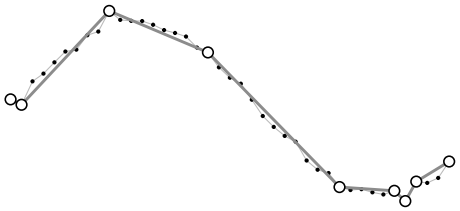


图 8
简化后的噪声样本

如同所有只看得到样本的方法，`fit_polyline` 限制的是与样本的偏差，而非与样本之间未知连续曲线的偏差；需要更贴近时请加密采样。

第 10 节　等值线

无异曲线、等产量曲线与等高线，都是二元函数的等值集。

10.1　等值集与切线

定义 36 (等值集) 对 $F:\mathbb{R}^2\to\mathbb{R}$ 与水平 $c\in\mathbb{R}$，等值集为

$$L_c=\{(x,y):F(x,y)=c\}.$$

对连续可微的 F，以 F_x、F_y 表示偏导数，$\nabla F=\left(F_x,F_y\right)$ 为梯度。在 $\nabla F\neq 0$ 的点附近，等值集是一条曲线，这由隐函数定理得出；以下引述所需的形式。

定理 37 (隐函数定理) *设 $\Omega\subset\mathbb{R}^2$ 为开集，$F:\Omega\to\mathbb{R}$ 连续可微，$p\in\Omega$，$F(p)=c$ 且 $F_y(p)\neq 0$。则存在开区间 I、J 满足 $p_x\in I$、$p_y\in J$、$I\times J\subset\Omega$，以及连续可微的 $\varphi:I\to J$、$\varphi(p_x)=p_y$，使得对 $(x,y)\in I\times J$，*

$$F(x,y)=c\quad\text{若且唯若}\quad y=\varphi(x).$$

此定理对任意个变量都成立，见 Rudin (1976) 的定理 9.28、Apostol (1974) 第二版的定理 13.7 与 Munkres (1991) 的定理 9.2。此处不予证明。

定理 38 (等值集的切线) *设 F 在开集 $\Omega\subset\mathbb{R}^2$ 上连续可微，$p\in\Omega$，$F(p)=c$ 且 $\nabla F(p)\neq 0$。则存在 p 的开邻域 N 与开区间 I 上的连续可微曲线 $\gamma:I\to\mathbb{R}^2$，满足 $\gamma(I)=L_c\cap N$、某个 $u_0\in I$ 使 $\gamma(u_0)=p$、对所有 $u\in I$ 有 $\gamma'(u)\neq 0$，且 $\gamma'(u)$ 平行于 $\gamma(u)$ 处的 $\left(F_y,-F_x\right)$。矢量 $\left(F_y,-F_x\right)$ 与 ∇F 正交。*

切线 $\left(F_y,-F_x\right)$ 不需除法，因此垂直切线（$F_y=0$）不需要另外处理；图形形式的斜率 $\mathrm{d}y/\,\mathrm{d}x=-F_x/F_y$ 在该处则为无穷大。

10.2　描绘

`trace_implicit` 不需解出 y 就能把等值集转为三次路径，因此可以处理会折返、封闭或分成数段的曲线。

`trace_implicit` `trace_implicit(⟨function⟩, *, levels, viewport, resolution=(101, 101), tolerance=1e-2, gradient=None)`

在矩形 `viewport` $= (x_{\min}, x_{\max}, y_{\min}, y_{\max})$ 内，描绘每个水平 c 的 $F(x,y) = c$，位于 `bezierkit.implicit`。

`function` F，以两个浮点数调用；在每个网格点都必须为有限值，否则抛出 `ValueError`。

`levels` 单一水平，或由有限水平值组成的可迭代对象。

`resolution` 沿 x 与 y 的网格点数，各至少 2。

(int, int) · 默认 `(101, 101)`

`tolerance` 简化与弯曲线段的容许偏差，单位同坐标。

float · 默认 `1e-2`

`gradient` $\nabla F = \left(F_x, F_y\right)$，以 tuple 或 `Vector` 回传。提供时，直线段会弯成沿曲线切线方向的三次曲线。

callable | *None* · 默认 `None`

算法是行进方格法，即 Lorensen & Cline (1987) 行进立方体法的二维模拟。网格单元的角点为 $v_{00}, v_{10}, v_{11}, v_{01}$，由左下角起逆时针排列，$F$ 的值分别为 $f_{00}, f_{10}, f_{11}, f_{01}$。角点的值不小于水平 c 时称为高。

定义 39 (边上的交点) 设网格的一条边由 p 到 q，且 $F(p) \geq c > F(q)$ 或 $F(q) \geq c > F(p)$。其交点为 $p + \lambda(q - p)$，其中

$$\lambda = \frac{c - F(p)}{F(q) - F(p)},$$

即 $F - c$ 沿该边的线性插值之零点。

一条边有交点，若且唯若它的两个角点高低不同。

1. 采样。在 viewport 的规则网格上计算 F。
2. 行进。在每个网格单元中找出四条边的交点（详见定义 39）。有两个交点的单元格贡献一条线段。有四个交点的单元格是鞍点：两个对角角点为高、另两个为低，由下述规则决定如何连接（参见图 9）。
3. 缝合。把端点相同的线段串成链。回到起点的链是封闭的；抵达 viewport 边界或分支点的链是开放的。互不相连的部分保持为不同的路径。
4. 简化。以 `tolerance` 对每条链调用 `fit_polyline`（详见第 9 节），不保留转角。
5. 弯曲（提供 `gradient` 时）。把每条直线段换成端点切线沿等值集方向的三次曲线（详见定理 38），控制柄长为弦长的三分之一。只有当三次曲线与弦的距离不超过 `tolerance` 时才采用；任一端 $\nabla F = 0$ 时维持直线。

鞍点规则使用四个角点值的双线性插值

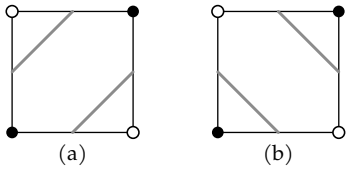


图 9
鞍点格子的两种中心

$$u(s,t) = (1-s)(1-t)f_{00} + s(1-t)f_{10} + stf_{11} + (1-s)tf_{01}, \quad (s,t) \in [0,1]^2.$$

命题 40 (双线性插值的中心值) 插值 u 在角点取角点值，在单元格中心的值等于四个角点值的平均：

$$u\left(\frac{1}{2}, \frac{1}{2}\right) = \frac{f_{00} + f_{10} + f_{11} + f_{01}}{4}.$$

此平均值不小于 c 时，套件将中心视为高。在鞍点单元格中，与中心同侧的两个角点在格内相连，另外两个角点各自被一条线段切开（参见图 9）。这就是中点决定法（midpoint decider）（Athawale & Johnson, 2019）。Nielson & Hamann (1991) 的渐近决定法（asymptotic decider）则比较 u 在其鞍点的值

$$u^* = \frac{f_{00}f_{11} - f_{10}f_{01}}{f_{00} - f_{10} - f_{01} + f_{11}},$$

两种规则只在 u^* 与平均值落在 c 两侧时才不同。

```
from bezierkit.implicit import trace_implicit

contours = trace_implicit(
    lambda x, y: x**2 * y,
    levels=[1, 2, 4],
    viewport=(0.5, 4, 0, 6),
    resolution=(121, 121),
    tolerance=0.01,
    gradient=lambda x, y: (2 * x * y, x**2),
)
print(contours.level_values)                    # (1.0, 2.0, 4.0)
print(len(contours.for_level(4)[0].segments))   # 15
```

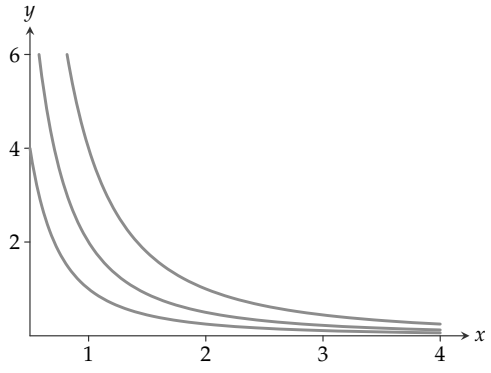


图 10
$x^2y = c$ 的三条曲线

10.3　结果

`ContourSet` `trace_implicit` 回传 `ContourSet`，其 `contours` 依给定顺序为每个水平保存一个 `LevelContours`。`level_values` 列出各水平，`paths` 汇集所有路径，`for_level(c)` 回传某一水平的路径（未描绘的水平抛出 `KeyError`）。

`LevelContours` 水平值 `level` 与路径 `paths`，每个连通部分一条 `PiecewiseBezier`。

10.4　限制

描绘出的曲线只在网格交点上精确；交点之间是行进方格法得到的折线，再于 `tolerance` 内简化与弯曲。需要更贴近时，请提高 `resolution` 并降低 `tolerance`。比网格单元小的特征可能遗漏，鞍点规则也只会选择两种可能连接方式之一。Leontief 无异曲线的折角等尖点会在网格间距内被圆滑化，因为网格看不到精确的转角。

第 11 节　采样与导出

导出器只写出几何数据，不选择颜色、线宽、主题、坐标轴、标签或画布，也绝不把三次曲线摊平成折线：TikZ 收到原生的 `.. controls ..` 曲线，SVG 收到原生的 `C` 命令。

11.1　采样

`UniformSampler` `UniformSampler(⟨整数⟩).sample(⟨曲线⟩)`
在曲线或路径的定义域上，以 `count` 个等距参数求值（含两端，`count` ≥ 2），位于 `bezierkit.sampling`。

`Sample` 采样的结果：唯读的参数 `t` 与 `points`，提供坐标数组 `x`、`y`，并可逐一取出 `(t, point)`。

```
from bezierkit.sampling import UniformSampler

sample = UniformSampler(5).sample(curve)
print(sample.x)   # [0.      0.90625 2.      3.09375 4.     ]
```

11.2　四舍五入与坐标变换

文本格式的导出器以固定的小数字数 `precision` 写出每个坐标，并接受 `transform`：在写出前把每个控制点（`Point`）对应到二维 `Point`，例如由数据坐标转为页面坐标。

命题 41 (四舍五入误差) *设 B、$\tilde{B}$ 是同为 n 次的 $\mathbb{R}^d$ 中 Bézier 曲线，控制点分别为 P_i 与 $\tilde{P}_i$，且对每个 i 与每个坐标 k 有 $|\tilde{P}_{i,k} - P_{i,k}| \le \varepsilon$。则对每个 $t \in [0,1]$ 与每个 k，*

$$|\tilde{B}_k(t) - B_k(t)| \le \varepsilon, \quad \|\tilde{B}(t) - B(t)\|_2 \le \sqrt{d}\,\varepsilon.$$

将每个坐标四舍五入到小数点后 p 位时，$\varepsilon = \tfrac{1}{2}\cdot 10^{-p}$。

因此在默认的 `precision=6` 下，导出的平面曲线处处与原曲线相差不超过 0.71×10^{-6}，不只在控制点上成立。依命题 8，仿射的 `transform` 是精确的；非仿射的变换能正确移动控制点，但一般不能正确移动控制点之间的曲线。

11.3　JSON

dumps　`dumps(⟨路径⟩, *, metadata=None, indent=None)`
位于 `bezierkit.export.json`。以表 4 的版本化格式写出路径，键依字母排序，不含 NaN 或无穷大。

loads　`loads(⟨字符串⟩)`
验证文档并回传 `PathDocument`，内含 `path` 与 `metadata`。格式或版本不明、线段不是恰好四个点、点的维度不符或数值非有限时，抛出 `ValueError`。

PathDocument　路径与调用端的 metadata，即 `loads()` 的回传值。

表 4
JSON 路径格式第 1 版

键	值
`schema`	字符串 `"bezierkit.path"`
`version`	整数 `1`；读取端拒绝其他版本
`dimension`	每个点的维度 d
`subpaths`	非空数组，元素为 `{"closed": bool, "segments": [...]}`，每个线段是四个含 d 个数值的点
`metadata`	JSON 对象，原样传递、不解读

```
from bezierkit.export.json import dumps, loads

text = dumps(path, metadata={"name": "arch"})
# {"dimension":2,"metadata":{"name":"arch"},"schema":"bezierkit.path",
#  "subpaths":[{"closed":false,"segments":[[[0.0,0.0],
[1.0,2.0],...]]}],"version":1}
assert loads(text).path.segments == path.segments
```

第 1 版已冻结：添加必要的键、改变封闭的语意或控制点的排列方式，都需要第 2 版。

11.4　SVG

to_svg_path_data　`to_svg_path_data(⟨路径⟩, *, precision=6, transform=None)`

from_svg_path_data　`from_svg_path_data(⟨字符串⟩)`
位于 `bezierkit.export.svg`。回传平面路径的路径数据字符串（即 `d` 属性），只使用绝对坐标的 `M`、`C` 与 `Z` 命令，每条子路径一个 `M`；不产生 SVG 文档或样式。`from_svg_path_data()` 解析同一子集；相对命令、直线与圆弧抛出 `ValueError`。

```
from bezierkit.export.svg import to_svg_path_data

print(to_svg_path_data(path, precision=2))   # M 0.00 0.00 C 1.00 2.00
3.00 2.00 4.00 0.00
```

11.5　TikZ

to_tikz　to_tikz(⟨路径⟩, *, precision=6, transform=None, options=None)

segment_to_tikz　segment_to_tikz(⟨*segment*⟩, *, precision=6, transform=None, include_start=True)

位于 bezierkit.export.tikz。每条子路径一个 \draw 命令，每个线段写成 .. controls (P1) and (P2) .. (P3)，封闭子路径以 -- cycle 结尾。options 原样放入 \draw[options]；默认不加任何选项。segment_to_tikz() 只写出一个线段、不含 \draw，供需要自行组合命令的调用端使用。

```
from bezierkit.export.tikz import to_tikz

print(to_tikz(path, precision=2, options="thick"))
# \draw[thick] (0.00,0.00) .. controls (1.00,2.00) and (3.00,2.00) ..
(4.00,0.00);
```

本手册的每张图都是这样产生的：脚本以 bezierkit 创建曲线，用 to_tikz() 写进 standalone LaTeX 文档，与坐标轴、标签放在一起，再编译成 PDF。每张图的 .tex 源文件都随手册提供。

11.6　Matplotlib

from_path　from_path(⟨路径⟩, *, transform=None)

to_path　to_path(⟨路径⟩)

approximate_path　approximate_path(⟨路径⟩, ⟨*transform*⟩, *, tolerance, max_depth=20)

位于 bezierkit.adapters.matplotlib，需安装 matplotlib 选用依赖。from_path() 精确转换 Matplotlib Path：MOVETO、LINETO、CURVE3、CURVE4 与 CLOSEPOLY 都转为三次曲线，直线与二次曲线依命题 22 升阶。仿射的 transform 直接套用在控制点上，结果精确（详见命题 8）；非仿射的变换抛出 ValueError。to_path() 反向转换，每个线段一个 CURVE4。

非仿射变换（例如对数坐标轴）不会把三次曲线对应到三次曲线。approximate_path() 是明确的选择：它细分每个变换后的线段，直到中点与弦的距离在 tolerance / 2 以内，再以 tolerance / 2 对这些点调用 fit_polyline。它不宣称结果精确。

第 12 节　命令行界面

安装 cli 可选依赖后即可使用 bezierkit 命令，在命令行计算、采样与构造曲线。默认输出表格，也可输出 JSON 或 CSV 给其他程序使用。

```
bezierkit evaluate --points "0,0" --points "1,2" --points "3,2" --
points "4,0" \
                   --t 0.5 --order 1 --json
# [{"t":0.5,"value":[4.5,0.0]}]
```

12.1　控制点

需要曲线的命令以重复的 `--points` 选项接收控制点，每个是以逗号分隔的坐标（`"1,2"`，三维则为 `"1,2,3"`）。省略 `--points` 时，命令从标准输入读取含 `control_points` 数组的 JSON 对象，也就是 `construct` 的输出，因此命令之间可以用管道串接。

12.2　命令

`evaluate`　`bezierkit evaluate --t ⟨数值⟩ [--points ...] [--order ⟨整数⟩] [--json]`

在每个 `--t` 计算曲线值，或其 `--order` 阶导数（默认 0）。每个值可为数字或含端点的范围 `start:stop:step`，例如 `0:1:0.25`；`--t` 可重复。

`sample`　`bezierkit sample [--points ...] [--count ⟨整数⟩] [--format table|csv|json] [--output ⟨路径⟩]`

以 `--count` 个（默认 50）等距参数采样，含两端，每列为 `t`、`x`、`y`（三维另有 `z`，更高维为 `c3`、`c4`……）。`--output` 写入文件而非标准输出。

`construct slopes` / `construct tangents`　`bezierkit construct slopes --start ⟨point⟩ --end ⟨point⟩ --start-slope ⟨浮点数⟩ --end-slope ⟨浮点数⟩ [--start-handle --end-handle]`

`bezierkit construct tangents --start ... --end ... --start-direction ⟨vector⟩ --end-direction ⟨vector⟩ [...]`

以 `PlanarSlopes` 或 `TangentDirections`（详见第 8 节）构造三次曲线，并以 JSON 打印控制点。

```
bezierkit construct slopes --start "0,5" --end "5,0" \
                           --start-slope -2 --end-slope -0.3 \
| bezierkit sample --count 3 --format csv
# t,x,y
# 0.0,0.0,5.0
# 0.5,2.3085202413545525,2.272345260462411
# 1.0,5.0,0.0
```

12.3　错误

无效输入（例如格式错误的点，或超出 $[0,1]$ 的参数）会在标准错误输出打印 `Error: ...`，退出状态为 1，不显示 traceback。在命令前加上全域选项 `--debug` 可改为显示 Python traceback。

附录 A　证明

本附录依各章出现的顺序，证明其中陈述的引理、命题、定理与推论。以下 $b_{i,n}$ 为定义 1 的 Bernstein 多项式，B 为定义 5 的 Bézier 曲线，次数为 n，控制点为 $P_0, \dots, P_n$。若某个结果在文献中也有证明，章节中会标明出处；此处的证明写成自足的形式，使手册不依赖某一版本的书。

A.1　Bernstein 基底与求值

引理 2 的证明：　第一式。依 j 的位置分别讨论。

- 若 $j<0$ 或 $j>n$，三个系数都为零，因为 $j>n$ 蕴含 $j-1>n-1$。
- 若 $j=0$，左边为 $1+0=1=\binom{n}{0}$。
- 若 $j=n$，左边为 $0+1=1=\binom{n}{n}$。
- 若 $1\le j\le n-1$，通分为 $j!(n-j)!$：

$$\binom{n-1}{j}+\binom{n-1}{j-1}=\frac{(n-1)!(n-j)}{j!(n-j)!}+\frac{(n-1)!j}{j!(n-j)!}=(n-1)!\frac{n}{j!(n-j)!}=\binom{n}{j}.$$

第二式。若 $i<1$ 或 $i>n$，两边都为零：$i=0$ 时左边含因子 $i=0$，且 $\binom{n-1}{-1}=0$；$i<0$ 与 $i>n$ 时每个系数都为零。若 $1\le i\le n$，

$$i\binom{n}{i}=i\frac{n!}{i!(n-i)!}=n\frac{(n-1)!}{(i-1)!(n-i)!}=n\binom{n-1}{i-1}.$$

□

命题 3 的证明：　次数不超过 n 的实系数多项式空间 Π_n 的维度为 $n+1$，一组基底为 $1,t,\dots,t^n$。$n+1$ 个多项式 $b_{0,n},\dots,b_{n,n}$ 都属于 Π_n，因此只需证明它们线性无关。设对所有 t 有 $\sum_{i=0}^{n}c_ib_{i,n}(t)=0$。对 $t\in(0,1)$ 除以 $(1-t)^n>0$，并令 $u=t/(1-t)\in(0,\infty)$：

$$\sum_{i=0}^{n}c_i\binom{n}{i}u^i=0.$$

有无穷多个根的 u 的多项式必为零多项式，所以每个 i 都有 $c_i\binom{n}{i}=0$。因 $\binom{n}{i}\neq0$，所有 $c_i=0$。□

命题 4 的证明：　当 $t\in[0,1]$ 时，t 与 $1-t$ 都非负，因此每个 $b_{i,n}(t)$ 都非负。由二项式定理，

$$\sum_{i=0}^{n}\binom{n}{i}t^i(1-t)^{n-i}=(t+(1-t))^n=1.$$

□

命题 6 的证明：　约定 $0^0=1$。$b_{i,n}(0)=\binom{n}{i}0^i$ 在 $i=0$ 时为 1，在 $i\ge1$ 时为 0。同理 $b_{i,n}(1)=\binom{n}{i}0^{n-i}$ 在 $i=n$ 时为 1，在 $i<n$ 时为 0。因此 $B(0)=\sum_i b_{i,n}(0)P_i=P_0$，$B(1)=\sum_i b_{i,n}(1)P_i=P_n$。□

推论 7 的证明：　由命题 4，$B(t)=\sum_i\lambda_iP_i$，其中 $\lambda_i=b_{i,n}(t)\ge0$ 且 $\sum_i\lambda_i=1$。由定义，点的凸组合位于这些点的凸包内。□

命题 8 的证明：　由 M 的线性与命题 4（$\sum_i b_{i,n}(t)=1$），

$$
\begin{aligned}
A(B(t)) &= M\sum_{i} b_{i,n}(t)P_i + v \\
&= \sum_{i} b_{i,n}(t)MP_i + \left(\sum_{i} b_{i,n}(t)\right)v \\
&= \sum_{i} b_{i,n}(t)(MP_i + v) = \sum_{i} b_{i,n}(t)A(P_i).
\end{aligned}
$$

□

定理 10 的证明：对 r 作归纳。$r=0$ 时，$P_i^{(0)} = P_i = b_{0,0}(t)P_i$。假设对 $r-1$ 与每个容许的 i 都成立。对 $0 \le i \le n-r$，i 与 $i+1$ 对 $r-1$ 都是容许的，因此

$$
\begin{aligned}
P_i^{(r)} &= (1-t)\sum_{j=0}^{r-1} b_{j,r-1}(t)P_{i+j} + t\sum_{j=0}^{r-1} b_{j,r-1}(t)P_{i+1+j} \\
&= \sum_{j=0}^{r} \Big[(1-t)b_{j,r-1}(t) + t b_{j-1,r-1}(t)\Big]P_{i+j},
\end{aligned}
$$

其中第二个和式以 $j+1 \to j$ 重新编号，两端由约定 $b_{-1,r-1} = b_{r,r-1} = 0$ 涵盖。对 $0 \le j \le r$，由引理 2，方括号为

$$
\binom{r-1}{j}t^j(1-t)^{r-j} + \binom{r-1}{j-1}t^j(1-t)^{r-j} = \binom{r}{j}t^j(1-t)^{r-j} = b_{j,r}(t).
$$

取 $r=n$、$i=0$，结论即为 $P_0^{(n)} = B(t)$。 □

引理 11 的证明：把 $b_{i,n}(t)$ 乘以 $1 = (1-t)+t$：

$$
b_{i,n}(t) = \binom{n}{i}t^i(1-t)^{n+1-i} + \binom{n}{i}t^{i+1}(1-t)^{n-i}.
$$

对 $0 \le i \le n$，$\binom{n+1}{i} = (n+1)/(n+1-i)\cdot\binom{n}{i}$ 且 $\binom{n+1}{i+1} = (n+1)/(i+1)\cdot\binom{n}{i}$，所以

$$
\binom{n}{i} = \frac{n+1-i}{n+1}\binom{n+1}{i} = \frac{i+1}{n+1}\binom{n+1}{i+1}.
$$

因此第一项是 $b_{i,n+1}(t)$ 的 $(n+1-i)/(n+1)$ 倍，第二项是 $b_{i+1,n+1}(t)$ 的 $(i+1)/(n+1)$ 倍。 □

命题 12 的证明：由引理 11，

$$
\begin{aligned}
B(t) &= \sum_{i=0}^{n}\left[\frac{n+1-i}{n+1}b_{i,n+1}(t) + \frac{i+1}{n+1}b_{i+1,n+1}(t)\right]P_i \\
&= \sum_{k=0}^{n}\frac{n+1-k}{n+1}b_{k,n+1}(t)P_k + \sum_{k=1}^{n+1}\frac{k}{n+1}b_{k,n+1}(t)P_{k-1},
\end{aligned}
$$

其中第一个和式取 $k=i$，第二个取 $k=i+1$。第一个和式的系数在 $k=n+1$ 为零，可延伸到 $k=n+1$；第二个和式同理可延伸到 $k=0$。合并 $b_{k,n+1}(t)$ 的系数即得 Q_k。 □

A.2　导数、反转与分割

引理 13 的证明： 依 i 的取值分别讨论。

- 若 $i<0$ 或 $i>n$，则 $b_{i,n}=0$，右边两项也为零。
- 若 $i=0$，$b_{0,n}(t)=(1-t)^n$ 的导数为 $-n(1-t)^{n-1}=n\big[b_{-1,n-1}(t)-b_{0,n-1}(t)\big]$。
- 若 $i=n$，$b_{n,n}(t)=t^n$ 的导数为 $nt^{n-1}=n\big[b_{n-1,n-1}(t)-b_{n,n-1}(t)\big]$。
- 若 $1\le i\le n-1$，由乘积法则，

$$b'_{i,n}(t)=\binom{n}{i}\big[it^{i-1}(1-t)^{n-i}-(n-i)t^i(1-t)^{n-i-1}\big].$$

  由引理 2，$i\binom{n}{i}=n\binom{n-1}{i-1}$；以 $n-i$ 代替 i 并利用 $\binom{n}{n-i}=\binom{n}{i}$，得 $(n-i)\binom{n}{i}=n\binom{n-1}{i}$。因此两项分别为 $nb_{i-1,n-1}(t)$ 与 $nb_{i,n-1}(t)$。

□

定理 14 的证明： 由线性与引理 13，

$$B'(t)=n\sum_{i=0}^{n}b_{i-1,n-1}(t)P_i-n\sum_{i=0}^{n}b_{i,n-1}(t)P_i.$$

第一个和式中 $i=0$ 的项为零；代入 $k=i-1$ 得 $\sum_{k=0}^{n-1}b_{k,n-1}(t)P_{k+1}$。第二个和式中 $i=n$ 的项为零。因此

$$B'(t)=n\sum_{k=0}^{n-1}b_{k,n-1}(t)(P_{k+1}-P_k).$$

□

推论 15 的证明： 由定理 14，B' 是控制点为 $D_k=n(P_{k+1}-P_k)$ $(k=0,\dots,n-1)$ 的 $n-1$ 次 Bézier 曲线。对它套用命题 6，得 $B'(0)=D_0$、$B'(1)=D_{n-1}$。 □

引理 16 的证明： 对 $0\le i\le n$，由 $\binom{n}{i}=\binom{n}{n-i}$，

$$b_{i,n}(1-t)=\binom{n}{i}(1-t)^it^{n-i}=\binom{n}{n-i}t^{n-i}(1-t)^i=b_{n-i,n}(t).$$

其他 i 两边都为零。 □

命题 17 的证明： 令 $j=n-i$ 重新编号，并利用引理 16：

$$\sum_{i=0}^{n}b_{i,n}(t)P_{n-i}=\sum_{j=0}^{n}b_{n-j,n}(t)P_j=\sum_{j=0}^{n}b_{j,n}(1-t)P_j=B(1-t).$$

□

定理 18 的证明： *左段。* 因 $1-cs=(1-s)+s(1-c)$，由二项式定理，

$$B(cs)=\sum_{i=0}^{n}\binom{n}{i}(cs)^i[(1-s)+s(1-c)]^{n-i}P_i$$

$$=\sum_{i=0}^{n}\sum_{k=0}^{n-i}\binom{n}{i}\binom{n-i}{k}c^i(1-c)^k s^{i+k}(1-s)^{n-i-k}P_i.$$

代入 $j=i+k$，则 $0\le i\le j\le n$ 且 $n-i-k=n-j$。$\binom{n}{i}\binom{n-i}{j-i}$ 与 $\binom{n}{j}\binom{j}{i}$ 都等于 $n!/(i!(j-i)!(n-j)!)$，故

$$B(cs)=\sum_{j=0}^{n}\binom{n}{j}s^j(1-s)^{n-j}\sum_{i=0}^{j}\binom{j}{i}c^i(1-c)^{j-i}P_i$$

$$=\sum_{j=0}^{n}b_{j,n}(s)P_0^{(j)}.$$

最后一步用定理 10 取 $r=j$、$i=0$：$P_0^{(j)}=\sum_{i=0}^{j}b_{i,j}(c)P_i$。这就是 L_j 的结论。

*右段。*令 $\overline{B}$ 为控制点 $\overline{P}_i=P_{n-i}$ 的曲线，由命题 17，$\overline{B}(u)=B(1-u)$；令 $\overline{c}=1-c$。对 $\overline{B}$ 套用左段，

$$\overline{B}(\overline{c}s)=\sum_{j=0}^{n}b_{j,n}(s)\overline{L}_j,$$

其中由引理 16 与代换 $l=j-i$，

$$\overline{L}_j=\sum_{i=0}^{j}b_{i,j}(1-c)P_{n-i}=\sum_{l=0}^{j}b_{l,j}(c)P_{n-j+l}=P_{n-j}^{(j)}=R_{n-j}.$$

倒数第二个等号是定理 10 取 $r=j$、$i=n-j$，此指标是容许的，因为 $n-j\le n-r$。因此再用一次引理 16，

$$B(c+(1-c)s)=\overline{B}(\overline{c}(1-s))=\sum_{j=0}^{n}b_{j,n}(1-s)R_{n-j}$$

$$=\sum_{j=0}^{n}b_{n-j,n}(s)R_{n-j}=\sum_{k=0}^{n}b_{k,n}(s)R_k.$$

□

*推论 19 的证明：*令 $a=t_0$、$e=t_1$，则 $0\le a<e\le 1$。由定理 18 取 $c=e$，左段是控制点为 $L_0,\dots,L_n$ 的 Bézier 曲线 $u\mapsto B(eu)$。对它再套用定理 18，取 $c'=a/e\in[0,1)$ 并保留右段：

$$s\mapsto B\Big(e\Big(\frac{a}{e}+\Big(1-\frac{a}{e}\Big)s\Big)\Big)=B(a+(e-a)s).$$

□

A.3　三次线段与路径

*命题 21 的证明：*展开三次 Bernstein 形式并依 t 的次方合并，

$$\begin{aligned} B(t) &= (1-t)^3P_0 + 3t(1-t)^2P_1 + 3t^2(1-t)P_2 + t^3P_3 \\ &= \big(1 - 3t + 3t^2 - t^3\big)P_0 + \big(3t - 6t^2 + 3t^3\big)P_1 + \big(3t^2 - 3t^3\big)P_2 + t^3P_3 \\ &= at^3 + bt^2 + ct + P_0, \end{aligned}$$

其中 a、b、c 如命题所述。

固定坐标 k。B_k 是多项式，在紧致区间 $[0,1]$ 上连续，因此在其上取得最小值与最大值。设 t^* 是取得其中一个的点。若 $t^* \in (0,1)$，则 B_k 在内点有极值且可微，故 $B_k'(t^*) = 0$（Fermat 定理）。分两种情形。

- B_k' 不恒为零。它是次数至多为 2 的多项式，至多有两个根；若 t^* 在 $(0,1)$ 内，它就是其中之一。每种情形都有 $t^* \in T_k$。
- B_k' 恒为零。则 B_k 为常数，其极值在 $t = 0$ 也取得，且 $0 \in T_k$。

因此 B_k 在 $[0,1]$ 上的最小值与最大值，等于它在有限集 $T_k \subset [0,1]$ 上的最小值与最大值，也就是命题中的 α_k 与 β_k，因为它们就是 B_k 在曲线上的下确界与上确界。由定义 20，边界框是各区间 $[\alpha_k, \beta_k]$ 的乘积：包含曲线的方框必包含每个值 $B_k(t)$，因而包含 α_k 与 β_k；而这些区间的乘积本身就包含曲线。 □

*命题 22 的证明：直线。*直线 $t \mapsto (1-t)P_0 + tP_3$ 是控制点为 $R_0 = P_0$、$R_1 = P_3$ 的一次 Bézier 曲线。由命题 12 取 $n = 1$，它是控制点为

$$Q_0 = P_0, \quad Q_1 = \frac{1}{2}P_0 + \frac{1}{2}P_3, \quad Q_2 = P_3$$

的二次曲线。再套用命题 12 取 $n = 2$，得到控制点为

$$\begin{aligned} S_0 &= Q_0 = P_0, \\ S_1 &= \frac{1}{3}Q_0 + \frac{2}{3}Q_1 = \frac{2}{3}P_0 + \frac{1}{3}P_3, \\ S_2 &= \frac{2}{3}Q_1 + \frac{1}{3}Q_2 = \frac{1}{3}P_0 + \frac{2}{3}P_3, \\ S_3 &= Q_2 = P_3 \end{aligned}$$

的三次曲线，亦即对 $i = 0, \ldots, 3$ 有 $S_i = P_0 + \frac{i}{3}(P_3 - P_0)$。

*二次曲线。*由命题 12 取 $n = 2$ 与控制点 P_0, C, P_3，三次控制点为

$$Q_0 = P_0, \quad Q_1 = \frac{1}{3}P_0 + \frac{2}{3}C, \quad Q_2 = \frac{2}{3}C + \frac{1}{3}P_3, \quad Q_3 = P_3,$$

即 $Q_1 = P_0 + \frac{2}{3}(C - P_0)$、$Q_2 = P_3 + \frac{2}{3}(C - P_3)$。

每一步都是关于 t 的恒等式，因此曲线与其参数化不变。 □

命题 25 的证明： 令 B_S、B_T 为控制点分别是 P_i 与 Q_i 的三次 Bézier 曲线，使 $S(u) = B_S((u - u_0 + h_S)/h_S)$、$T(u) = B_T((u - u_0)/h_T)$。在 $u = u_0$ 处，两个参数分别为 1 与 0。由命题 6，$S(u_0) = P_3$、$T(u_0) = Q_0$。由连锁律与推论 15，

$$S'(u_0)=\frac{B_S'(1)}{h_S}=\frac{3(P_3-P_2)}{h_S},\quad T'(u_0)=\frac{B_T'(0)}{h_T}=\frac{3(Q_1-Q_0)}{h_T}.$$

1. 由定义 23 取 $k=0$，C 在 u_0 为 C^0 若且唯若 $S(u_0)=T(u_0)$，即 $P_3=Q_0$。
2. 当 $P_3=Q_0$, 定义 23 取 $k=1$ 多出 $S'(u_0)=T'(u_0)$ 的条件, 两边除以 3 即所述等式。
3. 当 $P_3=Q_0$，由定义 24，接点为 G^1 若且唯若 $S'(u_0)\neq 0$、$T'(u_0)\neq 0$，且存在 $\lambda>0$ 使 $T'(u_0)=\lambda S'(u_0)$。两个切矢量皆非零，若且唯若 $P_3\neq P_2$ 且 $Q_1\neq Q_0$，此时

$$Q_1-Q_0=\left(\lambda\frac{h_T}{h_S}\right)(P_3-P_2).$$

   反之，若 $Q_1-Q_0=\mu(P_3-P_2)$ 且 $\mu>0$，取 $\lambda=\mu h_S/h_T>0$，即得 $T'(u_0)=\lambda S'(u_0)$。
4. 均匀参数化下，m 段路径的每段占 $1/m$，故 $h_S=h_T$，第 2 项的等式化为 $P_3-P_2=Q_1-Q_0$。

□

A.4　建构与插值

命题 26 的证明：对具所述控制点的曲线 B，由命题 6 得 $B(0)=P_0$、$B(1)=P_3$，由推论 15 得

$$B'(0)=3(P_1-P_0)=D_0,\quad B'(1)=3(P_3-P_2)=D_1.$$

再设 Γ 是任一次数至多为 3、满足 $\Gamma(0)=P_0$、$\Gamma(1)=P_3$、$\Gamma'(0)=D_0$、$\Gamma'(1)=D_1$ 的多项式曲线。对每个坐标套用命题 3，存在唯一的 $R_0,\ldots,R_3$ 使 $\Gamma(t)=\sum_i b_{i,3}(t)R_i$。对控制点 R_i 作上述计算，得

$$\Gamma(0)=R_0,\quad \Gamma(1)=R_3,\quad \Gamma'(0)=3(R_1-R_0),\quad \Gamma'(1)=3(R_3-R_2).$$

与四个条件比较，得 $R_0=P_0$、$R_3=P_3$、$R_1=P_0+D_0/3$、$R_2=P_3-D_1/3$。所以 R_i 就是所述的控制点，$\Gamma=B$。□

命题 28 的证明：令 $G(s)=H(x_0+hs)$。映射 $H\mapsto G$ 是次数至多为 3 的多项式集合上的双射，反映射为 $G\mapsto G((x-x_0)/h)$。由连锁律，$H(x_i)=f_i$ 与 $H'(x_i)=m_i$（$i=0,1$）等价于

$$G(0)=f_0,\quad G(1)=f_1,\quad G'(0)=hm_0,\quad G'(1)=hm_1.$$

由命题 26 取 $d=1$、$P_0=f_0$、$P_3=f_1$、$D_0=hm_0$、$D_1=hm_1$，恰有一个次数至多为 3 的多项式 G 满足这四个条件，其 Bernstein 系数为 $c_0,\ldots,c_3$。因此 H 存在、唯一，且等于 $\sum_i b_{i,3}((x-x_0)/h)c_i$。□

命题 29 的证明：在 $t=t_0$ 与 $t=t_1$, 线段分别于 $s=0$ 与 $s=1$ 求值, 由命题 6 得 $H(t_0)=P_0$、$H(t_1)=P_3$。由连锁律，$H'(t)=B'(s)/h$。由推论 15，$B'(0)=3(P_1-P_0)=hD_0$、$B'(1)=3(P_3-P_2)=hD_1$，故 $H'(t_0)=D_0$、$H'(t_1)=D_1$。证明中不需要 $h>0$。□

命题 *30* 的证明： 将线段写成 $(X(s), Y(s))$。X 的控制点为

$$x_0,\quad x_0+\frac{h}{3},\quad x_1-\frac{h}{3}=x_0+2\frac{h}{3},\quad x_1,$$

由命题 22（由 x_0 到 x_1 的直线），它们正是直线 $s\mapsto x_0+hs$ 的控制点，故 $X(s)=x_0+hs$。Y 的控制点为 $y_0, y_0+hm_0/3, y_1-hm_1/3, y_1$，即命题 28 中对数据 $y_i=f(x_i)$、$m_i=f'(x_i)$ 的系数 $c_0,\dots,c_3$。因此 $Y(s)=H(x_0+hs)$，线段为 $s\mapsto(x_0+hs, H(x_0+hs))$。□

定理 *31* 的证明：$x\in\{x_0,x_1\}$ 的情形是平凡的：第一个公式两边皆为零，对任何 ξ 都成立。设 $x_0<x<x_1$，并令

$$E=f-H,\quad w(t)=(t-x_0)^2(t-x_1)^2,\quad K=\frac{E(x)}{w(x)},\quad g=E-Kw.$$

则 $w(x)>0$，g 四阶连续可微，且 $g(x_0)=g(x)=g(x_1)=0$：E 与 w 在 x_0、x_1 为零，而 $g(x)=E(x)-Kw(x)=0$。此外 $g'(x_0)=g'(x_1)=0$：由定义 27，$E'(x_i)=f'(x_i)-H'(x_i)=0$，而 w 在每个 x_i 有二重根，故 $w'(x_i)=0$。

由 Rolle 定理（Burden & Faires (2011) 的定理 1.7），g' 在某个 $\xi_1\in(x_0,x)$ 与某个 $\xi_2\in(x,x_1)$ 为零。因此 g' 有四个相异零点 $x_0<\xi_1<\xi_2<x_1$。g' 连续且三阶可微，由广义 Rolle 定理（Burden & Faires (2011) 的定理 1.10），存在 $\xi\in(x_0,x_1)$ 使 $g^{(4)}(\xi)=0$。由于 H 的次数至多为 3，而 w 是首项系数为 1 的四次多项式，$H^{(4)}=0$、$w^{(4)}=24$，因此

$$0=g^{(4)}(\xi)=f^{(4)}(\xi)-24K,\quad \text{即}\quad K=\frac{f^{(4)}(\xi)}{24}.$$

故 $f(x)-H(x)=E(x)=Kw(x)$，这就是第一个公式。

关于界限：$(x-x_0)(x_1-x)$ 是两个和为 h 的非负数之积，至多为 $(h/2)^2$。因此 $w(x)\le h^4/16$，

$$|f(x)-H(x)|\le\frac{M}{24}\cdot\frac{h^4}{16}=\frac{Mh^4}{384}.$$

□

A.5　拟合与等值线

推论 *32* 的证明： 设 $[a,b]$ 是深度 k 的区间，则 $b-a=L/2^k$。在此区间上，Hermite 线段的每个坐标都是 C 同一坐标的三次 Hermite 插值多项式（详见命题 29 与命题 28），因此由定理 31，其误差不超过 $M(b-a)^4/384$。任一参数处的欧氏误差至多为最大坐标误差的 $\sqrt{d}$ 倍，因为对 $v\in\mathbb{R}^d$ 有 $\|v\|_2\le\sqrt{d}\|v\|_\infty$：事实上 $\|v\|_2^2=\sum_k v_k^2\le d\|v\|_\infty^2$。特别地，在量测参数上取最大值得到的实测误差满足

$$e\le\sqrt{d}M\frac{L^4}{384\cdot 2^{4k}}.$$

此值不超过 ε，若且唯若 $2^{4k}\ge\sqrt{d}ML^4/(384\varepsilon)$，也就是 $k\ge\frac{1}{4}\log_2\!\left(\sqrt{d}ML^4/(384\varepsilon)\right)$。每个 $k\ge k^*$ 都满足此式，所以该深度的每个区间都会被接受。

再设 `max_depth` $\geq k^*$ 且 `max_segments` $\geq 2^{k^*}$。区间只有在不合格时才会被分割，而这只发生在深度 $k < k^*$。因此所有区间的深度至多为 k^*，不会因深度而抛出 `ToleranceNotMet`。被接受的区间是把 $[t_0, t_1]$ 对半至多 k^* 次得到的互不重叠的片段，所以至多有 2^{k^*} 个。不合格的区间至少会产生两个被接受的线段，因此「目前已接受的线段数加二」不会超过这个最终数目，而它至多为 `max_segments`；所以线段数上限也不会被超过。 □

命题 34 的证明： 以 $\mathrm{rdp}(i, j)$ 表示算法对指针 $i < j$ 运行的递归步骤，它回传由 i 到 j 的递增指针列表。对 $j - i$ 作强归纳，证明对回传列表中任两个相邻指针 $u < v$，每个满足 $u \leq l \leq v$ 的指针 l 都有 $\mathrm{dist}(v_l, [v_u, v_v]) \leq \varepsilon$。

基底。 若 $j \leq i + 1$，列表为 (i, j)，且介于 i 与 j（含）之间的指针只有 i 与 j，其顶点就是弦的端点，距离为零。

归纳步骤。 否则令 δ^* 为严格介于 i 与 j 之间的顶点到 $\left[v_i, v_j\right]$ 的最大距离。若 $\delta^* \leq \varepsilon$，列表为 (i, j)，由 δ^* 的选取以及端点距离为零，结论成立。若 $\delta^* > \varepsilon$，步骤选取指针 m，$i < m < j$，并回传 $\mathrm{rdp}(i, m)$ 的列表接上 $\mathrm{rdp}(m, j)$ 的列表，m 只出现一次。结果中任两个相邻指针，在这两个列表之一中也是相邻的。由于 $m - i$ 与 $j - m$ 都小于 $j - i$，归纳假设适用于两者。

步骤 2 与 3 对步骤 2 保留的每两个相邻顶点运行 `rdp` 并串接结果。输出恰保留回传的指针 $a_0 < \ldots < a_m$，其中每两个相邻指针在某个列表中也相邻。这就证明了不等式。输出线段的 `fit_error` 是 $\max_{a_j \leq i \leq a_{j+1}} \mathrm{dist}\Big(v_i, \left[v_{a_j}, v_{a_{j+1}}\right]\Big)$，因此不超过 ε。 □

推论 35 的证明：

1. 到凸集的距离是凸函数。对线段 $K = [a, b]$，设 x_1, x_2 满足 $\mathrm{dist}(x_i, K) \leq \varepsilon$，并在 $y_i \in K$ 取得（由紧致性，最小值存在），$\lambda \in [0, 1]$。则 $\lambda y_1 + (1 - \lambda) y_2 \in K$，由三角不等式，

$$\|\lambda x_1 + (1 - \lambda) x_2 - (\lambda y_1 + (1 - \lambda) y_2)\|_2 \leq \lambda \|x_1 - y_1\|_2 + (1 - \lambda) \|x_2 - y_2\|_2 \leq \varepsilon.$$

所以 $\mathrm{dist}(\lambda x_1 + (1 - \lambda) x_2, K) \leq \varepsilon$。

折线上的点 x 位于某条边 $[v_i, v_{i+1}]$ 上，即 $x = \lambda v_i + (1 - \lambda) v_{i+1}$。选取 j 使 $a_j \leq i < a_{j+1}$；因指针为整数，$i + 1 \leq a_{j+1}$，由命题 34，v_i 与 v_{i+1} 都在 $\left[v_{a_j}, v_{a_{j+1}}\right]$ 的 ε 范围内。由刚证的凸性，x 亦然。

2. 通过步骤 1 的点就是某个 v_i，由命题 34，它离某条弦不超过 ε。被删除的输入点 p 与其前一个保留的顶点（记为 v）相距不超过 δ；设弦 $[a, b]$ 与 v 相距不超过 ε，则

$$\mathrm{dist}(p, [a, b]) \leq \|p - v\|_2 + \mathrm{dist}(v, [a, b]) \leq \delta + \varepsilon.$$

封闭折线结尾重复的起点在同样条件下被删除，取 $v = v_0$ 即可。 □

命题 40 的证明： 把 $[0, 1]^2$ 的四个角代入 u，每个角只有一项不为零且系数为 1，所以 u 在角点取角点值。在 $s = t = 1/2$，四个权重乘积都等于 $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$，因此 $u\left(\frac{1}{2}, \frac{1}{2}\right) = \frac{1}{4}(f_{00} + f_{10} + f_{11} + f_{01})$。 □

定理 38 的证明： 因为 $\nabla F(p) \neq 0$，$F_y(p) \neq 0$ 或 $F_x(p) \neq 0$。

情形 $F_y(p) \neq 0$。套用定理 37：存在 I、J 与 C^1 函数 $\varphi : I \to J$，$\varphi(p_x) = p_y$，且 $L_c \cap (I \times J) = \{(x, \varphi(x)) : x \in I\}$。令 $N = I \times J$、$\gamma(x) = (x, \varphi(x))$。对 $F(x, \varphi(x)) = c$ 用连锁律微分，在 $\gamma(x)$ 处得 $F_x + F_y\varphi' = 0$。由 F_y 的连续性，可缩小 I 与 J 使 $F_y \neq 0$ 在 N 上成立；于是

$$\gamma'(x) = (1, \varphi'(x)) = \left(1, -\frac{F_x}{F_y}\right) = \frac{1}{F_y}\big(F_y, -F_x\big),$$

它非零，且在 $\gamma(x)$ 处平行于 $\big(F_y, -F_x\big)$。

情形 $F_x(p) \neq 0$。交换 x 与 y 的角色：存在 C^1 函数 ψ 使 $L_c \cap N = \{(\psi(y), y)\}$，曲线 $\gamma(y) = (\psi(y), y)$，且 $\psi' = -F_y/F_x$，故

$$\gamma'(y) = \left(-\frac{F_y}{F_x}, 1\right) = -\frac{1}{F_x}\big(F_y, -F_x\big),$$

同样非零且平行于 $\big(F_y, -F_x\big)$。

最后 $\nabla F \cdot \big(F_y, -F_x\big) = F_xF_y - F_yF_x = 0$。 □

A.6　导出

命题 41 的证明： 两条曲线的差为

$$\tilde{B}(t) - B(t) = \sum_{i=0}^{n} b_{i,n}(t)\big(\tilde{P}_i - P_i\big).$$

由命题 4，权重 $b_{i,n}(t)$ 非负且和为 1，因此对每个坐标，

$$|\tilde{B}_k(t) - B_k(t)| \leq \sum_{i=0}^{n} b_{i,n}(t)\,|\tilde{P}_{i,k} - P_{i,k}| \leq \varepsilon.$$

由于对 $v \in \mathbb{R}^d$ 有 $\|v\|_2 \leq \sqrt{d}\|v\|_\infty$（见 Golub & Van Loan (2013) 第 2.2 节），得 $\big\|\tilde{B}(t) - B(t)\big\|_2 \leq \sqrt{d}\varepsilon$。

导出器以 Python 的定点格式写出每个坐标的 p 位小数，得到最接近该坐标的 10^{-p} 的整数倍，与原坐标相差至多 $\frac{1}{2}\cdot 10^{-p}$。导出器也把绝对值小于 $\frac{1}{2}\cdot 10^{-p}$ 的坐标写成 0，改变量即该绝对值，同样至多为 $\frac{1}{2}\cdot 10^{-p}$。 □

更新纪录

本章列出影响软件包使用的版本变更，不含仅修改文档的版本；各条目按 l3doc 惯例标注于它所描述的功能旁，页码即该功能的实际页码。

v1.0.0 (2026-10-04)

一般：第一个正式版；公开 API 按语义化版本管理 ····· 2

`bezierkit`：持续整合测试 Python 3.10、3.11、3.12 与 3.13 ····· 3

`PiecewiseBezier.segment`：`t0` 落在线段内部时回传正确的区间；先前会先在 `t0` 分割再对后半段重新参数化，导致终点错误 ····· 14

v0.5.0rc1 (2026-09-28)

一般：首次发布到 PyPI：任意次数的 Bézier 曲线、三次线段与路径、构造、Hermite 插值、拟合、等值线描绘、采样、导出器与命令行界面 ····· 2

命令索引

A

approximate_path 26

B

BernsteinBasis 7

BernsteinEvaluator 9

BezierCurve 7

BezierCurve.derivative 10

BezierCurve.reversed 10

BezierCurve.segment 11

BezierCurve.split 11

C

construct slopes 27

construct tangents 27

ContourSet 24

CubicBezierSegment 11

CubicBezierSegment.bounding_box 12

D

DeCasteljauEvaluator 8

dumps 25

E

EndpointDerivatives 15

evaluate 27

F

fit_graph 19

fit_parametric 18

fit_polyline 20

from_path 26

from_svg_path_data 25

G

graph_hermite 17

H

hermite_error_bound 18

I

Interval 5

L

LevelContours 24

line_to_cubic 13

loads 25

M

maximum_polyline_deviation 20

P

ParameterValues 5

parametric_hermite 16

PathDocument 25

PiecewiseBezier 13

PlanarSlopes 16

Point 5

PointSet 5

Q

quadratic_to_cubic 13

S

Sample 24

sample 27

segment_to_tikz 26

T

TangentDirections 15

to_cubic 13

to_path 26

to_svg_path_data 25

to_tikz 26

trace_implicit 22

U

UniformSampler 24

V

Vector 5

参考文献

Apostol, T. M. (1974). *Mathematical Analysis* (2nd ed.). Addison-Wesley.

Athawale, T., & Johnson, C. R. (2019). Probabilistic Asymptotic Decider for Topological Ambiguity Resolution in Level-Set Extraction for Uncertain 2D Data. *IEEE Transactions on Visualization and Computer Graphics*, *25*(1), 1163–1172. https://doi.org/10.1109/TVCG.2018.2864505

Barsky, B. A., & DeRose, T. D. (1989). Geometric Continuity of Parametric Curves: Three Equivalent Characterizations. *IEEE Computer Graphics and Applications*, *9*(6), 60–69. https://doi.org/10.1109/38.41470

Bernstein, S. N. (1912). Démonstration du théorème de Weierstrass fondée sur le calcul des probabilités. *Communications De La Société Mathématique De Kharkov*, *13*, 1–2.

Bézier, P. (1966). Définition numérique des courbes et surfaces I. *Automatisme*, *11*, 625–632.

Burden, R. L., & Faires, J. D. (2011). *Numerical Analysis* (9th ed.). Brooks/Cole.

Douglas, D. H., & Peucker, T. K. (1973). Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or Its Caricature. *Cartographica*, *10*(2), 112–122. https://doi.org/10.3138/FM57-6770-U75U-7727

Farin, G. (2002). *Curves and Surfaces for CAGD: A Practical Guide* (5th ed.). Morgan Kaufmann.

Farouki, R. T. (2012). The Bernstein Polynomial Basis: A Centennial Retrospective. *Computer Aided Geometric Design*, *29*(6), 379–419. https://doi.org/10.1016/j.cagd.2012.03.001

Floater, M. S. (2025,). *An Introduction to Spline Theory*. https://www.uio.no/studier/emner/matnat/math/MAT4170/v25/undervisningsmateriale/spline_notes.pdf

Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.

Hershberger, J., & Snoeyink, J. (1994). An O(n \log n) Implementation of the Douglas–Peucker Algorithm for Line Simplification. *Proceedings of the Tenth Annual Symposium on Computational Geometry*, 383–384. https://doi.org/10.1145/177424.178097

Lorensen, W. E., & Cline, H. E. (1987). Marching Cubes: A High Resolution 3D Surface Construction Algorithm. *ACM SIGGRAPH Computer Graphics*, *21*(4), 163–169. https://doi.org/10.1145/37402.37422

Müller, A. (2024). A Tour d'Horizon of de Casteljau's Work. *Computer Aided Geometric Design*, *113*, 102366. https://doi.org/10.1016/j.cagd.2024.102366

Munkres, J. R. (1991). *Analysis on Manifolds*. Addison-Wesley.

Nielson, G. M., & Hamann, B. (1991). The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes. *Proceedings of Visualization '91*, 83–91. https://doi.org/10.1109/VISUAL.1991.175782

Prautzsch, H., Boehm, W., & Paluszny, M. (2002). *Bézier and B-Spline Techniques*. Springer. https://doi.org/10.1007/978-3-662-04919-8

Ramer, U. (1972). An Iterative Procedure for the Polygonal Approximation of Plane Curves. *Computer Graphics and Image Processing*, *1*(3), 244–256. https://doi.org/10.1016/S0146-664X(72)80017-0

Rudin, W. (1976). *Principles of Mathematical Analysis* (3rd ed.). McGraw-Hill.

Stoer, J., & Bulirsch, R. (2002). *Introduction to Numerical Analysis* (3rd ed.). Springer. https://doi.org/10.1007/978-0-387-21738-3